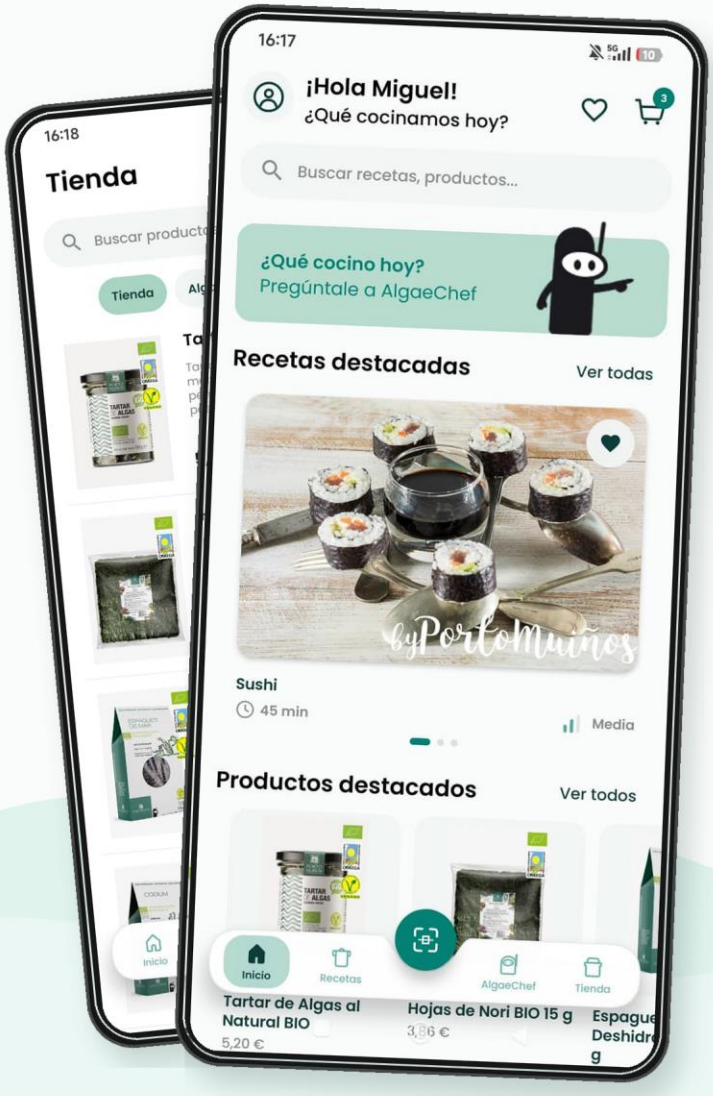
PORTO·MUIÑOS®
LAS VERDURAS DEL MAR

PORTO-MUIÑOS APP

Proyecto Fin de Ciclo

Desarrollo de Aplicaciones Multiplataforma

Aplicación móvil para descubrir, aprender y cocinar con algas.





Miguel Fernández González
2º DAM Dual · Curso 2025/2026
CIFP A Carballeira - Marcos Valcárcel
Ourense · Septiembre de 2026

Resumen

El presente Proyecto de Fin de Ciclo consiste en el diseño y desarrollo de una aplicación móvil Android orientada a facilitar el conocimiento, la utilización y el consumo de los productos de Porto-Muiños, empresa gallega especializada en algas y otros productos vinculados al medio marino. La propuesta parte de un reto planteado en el programa InnovaTech FP y evoluciona hasta una implementación individual, funcional y conectada a un backend real. La aplicación ha sido desarrollada con Kotlin y Jetpack Compose, utilizando Supabase como plataforma de backend para autenticación, base de datos, almacenamiento y servicios asociados. Entre las principales funcionalidades implementadas se encuentran el registro e inicio de sesión, autenticación mediante Google OAuth, recuperación de contraseña, catálogo de productos, recetario, búsqueda y filtrado, favoritos, carrito persistente por usuario, lectura de códigos QR, navegación entre productos y recetas relacionadas y conexión con la tienda oficial de Porto-Muiños. También se incluye AlgaeChef, un demostrador funcional de asistente culinario basado en recomendaciones controladas enlazadas con contenido real de la aplicación. El proyecto adopta una arquitectura MVVM apoyada en el patrón Repository y aplica mecanismos de seguridad mediante autenticación y políticas Row Level Security. La solución ha sido validada mediante pruebas unitarias y funcionales, inspección estática, emulador Android y dispositivo físico, obteniendo como resultado una aplicación estable, mantenible y preparada para futuras ampliaciones.

Abstract

This Final Project focuses on the design and development of an Android mobile application aimed at helping users discover, understand and use Porto-Muiños products, a Galician company specialized in seaweed and other marine-related food products. The project originated from a challenge proposed within the InnovaTech FP programme and later evolved into an individual, functional implementation connected to a real backend. The application was developed using Kotlin and Jetpack Compose, with Supabase providing authentication, database, storage and related backend services. The main implemented features include user registration and login, Google OAuth

authentication, password recovery, product catalogue, recipe collection, search and filtering, favourites, a persistent user-specific shopping cart, QR code scanning, navigation between related products and recipes, and integration with the official Porto-Muiños online store. The project also includes AlgaeChef, a functional prototype of a culinary assistant based on controlled recommendations linked to real application content. The application follows an MVVM architecture supported by the Repository pattern and uses authentication and Row Level Security policies to protect user data. The solution was validated through unit and functional testing, static code inspection, Android emulation and testing on a physical device. The final result is a stable and maintainable mobile application that provides a solid foundation for future extensions.

Índice

Resumen 1
Abstract 1
Índice 3
1. Introducción y contextualización 7
1.1. Origen del proyecto 7
1.2. Motivación y justificación 7
1.3. Problema detectado 8
1.4. Finalidad del proyecto 8
1.5. Alcance del proyecto 9
1.6. Limitaciones del alcance 10
1.7. Identidad visual 11
2. Objetivos 11
2.1. Objetivo general 11
2.2. Objetivos específicos 11
2.3. Grado de cumplimiento 12
3. Estudio de mercado y propuesta de valor 13
3.1. Público objetivo 13
3.2. Estado actual y soluciones existentes 14
3.3. Competidores y alternativas 15
3.4. Diferenciación de Porto-Muiños App 15
3.5. Viabilidad de la propuesta 16
4. Análisis y requisitos 17
4.1. Requisitos funcionales 17
4.2. Requisitos no funcionales 17
4.3. Actores del sistema 18
4.4. Casos de uso 18
4.5. Diagrama de casos de uso 19
5. Planificación y metodología de desarrollo 20
5.1. Fases del proyecto 20
5.2. Priorización funcional 21
5.3. Evolución del proyecto 22
5.4. Gestión de riesgos y alcance 22
6. Presupuesto y viabilidad económica 23
6.1. Recursos humanos 23
6.2. Coste estimado del desarrollo 23
6.3. Herramientas y servicios 24
6.4. Infraestructura 24
6.5. Presupuesto total estimado 25
7. Diseño de la solución 26
7.1. Diseño de interfaz y experiencia de usuario 26
7.2. Sistema visual 26
7.3. Flujo de navegación 29
7.4. Wireframes y prototipos 31
7.5. Adaptación a diferentes configuraciones Android 33

8. Arquitectura y diseño técnico 33
8.1. Arquitectura general del sistema 33
8.2. Patrón MVVM 34
8.3. Patrón Repository 35
8.4. Separación de responsabilidades 36
8.5. Gestión del estado 36
8.6. Navegación 36
8.7. Estructura de paquetes y carpetas 37
8.8. Componentes reutilizables 37
9. Modelo de datos y backend 37
9.1. Arquitectura de Supabase 37
9.2. Modelo Entidad-Relación 38
9.3. Diagrama E/R 39
9.4. Descripción de tablas 41
9.4.1. auth.users 41
9.4.2. products 41
9.4.3. recipes 42
9.4.4. recipe_products 43
9.4.5. favorite_products 43
9.4.6. favorite_recipes 44
9.4.7. cart_items 44
9.5. Relaciones, cardinalidades e integridad referencial 45
9.6. Row Level Security 46
9.7. Supabase Auth 46
9.8. Google OAuth 46
9.9. Storage 47
9.10. Edge Function delete-account 47
9.11. Gestión de credenciales 47
10. Tecnologías, herramientas, APIs y librerías 48
10.1. Kotlin 48
10.2. Jetpack Compose 48
10.3. Android Studio 48
10.4. Gradle y JDK 21 48
10.5. Supabase 49
10.6. PostgreSQL 49
10.7. CameraX 49
10.8. ML Kit 49
10.9. Coil 49
10.10. Kotlin Serialization y Ktor 50
10.11. Figma 50
10.12. Justificación de las tecnologías seleccionadas 50
11. Desarrollo e implementación 51
11.1. Autenticación 51
11.2. Recuperación de contraseña y deep links 52
11.3. Google OAuth 53
11.4. Catálogo de productos 53
11.5. Recetario 55
11.6. Buscadores y filtros 56

11.7. Relaciones producto-receta 56
11.8. Favoritos 57
11.9. Carrito 59
11.10. Integración con la tienda oficial 60
11.11. Escáner QR 61
11.12. AlgaeChef 62
11.13. Perfil 64
11.14. Persistencia y sincronización de datos 65
12. Calidad del código y buenas prácticas 66
12.1. Separación de responsabilidades 66
12.2. Nomenclatura y estilo 66
12.3. Comentarios y documentación interna 67
12.4. Refactorización a MVVM 67
12.5. Eliminación de código y recursos obsoletos 67
12.6. Auditoría con Android Studio 68
12.7. Seguridad 68
13. Pruebas y validación 69
13.1. Estrategia de pruebas 69
13.2. Pruebas unitarias 69
13.3. Pruebas funcionales 70
13.4. Pruebas de autenticación 70
13.5. Pruebas de persistencia multiusuario 71
13.6. Pruebas del QR 71
13.7. Pruebas en emulador 72
13.8. Pruebas en dispositivo físico 72
13.9. Pruebas de interfaz y modos de navegación 72
13.10. Inspección estática 73
13.11. Resultados 73
14. Problemas encontrados y soluciones adoptadas 73
14.1. Configuración de Supabase 73
14.2. Autenticación y deep links 74
14.3. Google OAuth 74
14.4. Lectura QR 75
14.5. Persistencia de favoritos y carrito 75
14.6. Sincronización rápida del carrito 75
14.7. Adaptación a navegación por gestos y tres botones 76
14.8. Crecimiento de PortoMuinosApp.kt 76
14.9. Refactorización a MVVM 76
14.10. Gestión del alcance de AlgaeChef 77
15. Innovación y aspectos destacables 77
15.1. Integración del ecosistema producto-receta-compra 78
15.2. QR como puente físico-digital 78
15.3. Personalización por usuario 78
15.4. AlgaeChef como concepto de asistente culinario 78
15.5. Integración de tecnologías cloud en una aplicación Android 79
16. Conclusiones 79
16.1. Resultados obtenidos 80

16.2. Cumplimiento de objetivos 80
16.3. Aprendizajes técnicos 80
16.4. Valoración personal del proyecto 81
17. Líneas futuras y propuestas de mejora 81
17.1. AlgaeChef con IA generativa real 82
17.2. Preferencias alimentarias 82
17.3. Alergias e intolerancias 82
17.4. Autenticación Apple 83
17.5. Contraseña para cuentas OAuth 83
17.6. Perfil sincronizado en backend 83
17.7. Compra integrada 83
17.8. Notificaciones 84
17.9. Internacionalización 84
17.10. Panel de administración y gestión de contenidos 84
18. Bibliografía y webgrafía 85
Anexo A. Manual técnico e instalación 87
Anexo B. Manual de usuario 94
Anexo C. Código fuente y aplicación 104

1. Introducción y contextualización

1.1. Origen del proyecto

El presente Proyecto de Fin de Ciclo consiste en el diseño y desarrollo de una aplicación móvil Android orientada a facilitar el conocimiento, la utilización y el consumo de los productos de Porto-Muiños, empresa gallega especializada en la comercialización de algas y otros productos vinculados al medio marino.

La idea tiene su origen en un reto propuesto por Porto-Muiños dentro del programa InnovaTech FP. El planteamiento inicial buscaba crear una solución digital capaz de acercar el potencial culinario de las algas al consumidor mediante información práctica, recomendaciones, recetas, combinaciones con otros alimentos y conexiones con canales de compra. Durante aquella fase se elaboró una propuesta conceptual y un prototipo de interfaz que sirvieron como punto de partida para definir la experiencia de usuario y los principales flujos de navegación.

A partir de esa base conceptual, el presente proyecto desarrolla de forma individual una implementación técnica propia, redefiniendo el alcance y la tecnología utilizada. El proyecto deja de ser únicamente una propuesta de producto para convertirse en una aplicación Android funcional, conectada a un backend real y validada tanto en emulador como en un dispositivo físico.

El resultado final se ha desarrollado con Kotlin y Jetpack Compose, utilizando Supabase como plataforma de backend. La aplicación integra autenticación, catálogo de productos, recetario, búsqueda, favoritos, carrito, escaneo de códigos QR, conexión con la tienda oficial y un demostrador denominado AlgaeChef, orientado a representar el funcionamiento de un futuro asistente culinario inteligente.

1.2. Motivación y justificación

Las algas poseen un importante potencial gastronómico, pero para una parte del público continúan siendo ingredientes poco habituales. La principal barrera no es necesariamente su disponibilidad, sino el desconocimiento sobre cómo utilizarlas en la cocina diaria: qué producto elegir, cómo prepararlo, con qué alimentos combinarlo o qué receta puede realizarse a partir de él.

Porto-Muiños dispone de un catálogo especializado y de un amplio conocimiento gastronómico asociado a estos productos. El proyecto parte de la oportunidad de acercar ese conocimiento al usuario mediante una experiencia móvil sencilla, visual y contextual, reduciendo la distancia existente entre descubrir un producto, comprender su uso, encontrar inspiración culinaria y acceder posteriormente a su compra.

La elección de una aplicación móvil resulta especialmente adecuada porque permite acompañar al usuario en situaciones reales de uso: durante la compra, al consultar un producto físico, mientras cocina o cuando desea decidir qué preparar con los ingredientes disponibles. De esta forma, la solución no se plantea únicamente como un catálogo comercial, sino como una capa digital que relaciona información, inspiración y acción.

1.3. Problema detectado

El problema principal identificado es la existencia de una barrera de conocimiento entre el consumidor y el uso cotidiano de las algas. Aunque el usuario pueda mostrar curiosidad por estos productos, esa curiosidad no siempre se transforma en una decisión de compra o de cocina si no dispone de referencias claras sobre preparación, combinaciones y recetas.

También existe una fragmentación habitual entre contenido gastronómico y producto. Una receta puede encontrarse en un canal, la información comercial en otro y la compra en un tercero. Esta separación obliga al usuario a realizar búsquedas adicionales y dificulta que el proceso de descubrimiento desemboque de forma natural en una acción concreta.

La propuesta intenta reducir ambas barreras integrando en un mismo entorno móvil el catálogo, las recetas, las relaciones entre ambos contenidos, la identificación mediante QR, la personalización básica mediante favoritos y carrito y la conexión con la tienda oficial de Porto-Muiños.

1.4. Finalidad del proyecto

La finalidad del proyecto es ofrecer una experiencia móvil coherente que ayude al usuario a comprender mejor los productos de Porto-Muiños y a incorporarlos con mayor

facilidad a su cocina. La aplicación pretende convertir la consulta de información en un recorrido útil: descubrir, entender, cocinar, guardar y, cuando corresponda, acceder a la compra.

Para ello, uno de los principios funcionales más importantes es la relación bidireccional entre productos y recetas. Desde una receta es posible identificar los productos de Porto-Muiños utilizados y acceder a sus fichas, mientras que desde un producto pueden consultarse recetas relacionadas. Este modelo evita pantallas aisladas y mantiene el contexto del usuario durante la navegación.

1.5. Alcance del proyecto

El alcance del proyecto se concentra en la construcción de una aplicación Android funcional y demostrable. Se han priorizado las funcionalidades que permiten recorrer el flujo principal de la solución y validar su propuesta de valor con datos, autenticación y persistencia reales.

La versión desarrollada permite registrar y autenticar usuarios, mantener la sesión, recuperar la contraseña, iniciar sesión mediante Google, consultar productos y recetas almacenados en Supabase, realizar búsquedas y filtros, gestionar favoritos y un carrito persistente por cuenta, escanear códigos QR, navegar entre productos y recetas relacionadas y acceder desde la aplicación a la tienda oficial para completar la compra.

AlgaeChef se incluye como un demostrador funcional del concepto de asistente culinario. La interacción está implementada y las respuestas se vinculan con contenido real de la aplicación, pero en esta versión no se ha integrado un modelo de inteligencia artificial generativa externo. Esta decisión permite demostrar el flujo de uso y su integración con el resto del producto sin añadir una dependencia tecnológica de mayor complejidad en la fase final del proyecto.

Bloque funcional	Estado	Observación
Autenticación por correo	**Implementado**	Registro, confirmación, login, logout y recuperación de contraseña.
Google OAuth	**Implementado**	Acceso mediante cuenta de Google a través de Supabase Auth.
Catálogo y recetario	**Implementado**	Contenido remoto almacenado en Supabase.
Búsqueda y filtros	**Implementado**	Búsqueda interna de productos y recetas.
Favoritos	**Implementado**	Persistencia por usuario mediante RLS.
Carrito	**Implementado**	Persistencia por usuario y enlace con la tienda oficial.
Escáner QR	**Implementado**	Lectura con cámara y apertura de la ficha interna del producto.
AlgaeChef	**Demostrador funcional**	Recomendaciones controladas enlazadas con recetas reales.
Inicio con Apple	**No implementado**	Se mantiene como ampliación futura.
Preferencias y alergias	**No implementado**	Previstas como evolución futura del perfil.
IA generativa externa	**Fuera del alcance**	Propuesta de mejora para una versión posterior.

Tabla 1. Alcance funcional de la versión desarrollada del proyecto.

1.6. Limitaciones del alcance

El proyecto no pretende sustituir la infraestructura comercial existente de Porto-Muiños ni desarrollar una plataforma completa de comercio electrónico. El carrito de la aplicación funciona como herramienta de selección y persistencia, mientras que el pago y el envío se completan en la tienda oficial de la empresa.

También quedan fuera de esta versión un módulo de noticias, la autenticación mediante Apple, la gestión avanzada de preferencias alimentarias y alergias, la sincronización remota de una imagen de perfil y la incorporación de un modelo generativo real en AlgaeChef. Estos elementos se mantienen como líneas de evolución

coherentes con el concepto inicial, pero no forman parte del alcance técnico cerrado para el proyecto.

1.7. Identidad visual

La interfaz se ha diseñado buscando coherencia con la identidad visual de Porto-Muiños. Se emplean tonalidades verdes y turquesas asociadas a la marca y al entorno marino, superficies claras y una composición limpia que prioriza la lectura y la fotografía de producto. La tipografía principal de la aplicación es Poppins, utilizada en distintos pesos para establecer jerarquías visuales.

El logotipo oficial de Porto-Muiños se utiliza en las pantallas de acceso y en la identidad del launcher de Android. La mascota AlgaeChef aporta un elemento reconocible y cercano en los flujos relacionados con el asistente y determinadas pantallas de apoyo. El objetivo no es rediseñar la marca, sino trasladar su lenguaje visual a una interfaz móvil contemporánea.

2. Objetivos

2.1. Objetivo general

Diseñar y desarrollar una aplicación móvil Android funcional que facilite al usuario el descubrimiento y la utilización culinaria de los productos de Porto-Muiños, integrando catálogo, recetas, identificación mediante códigos QR, personalización básica y conexión con los canales de compra de la empresa dentro de una experiencia visual coherente, segura y fácil de utilizar.

2.2. Objetivos específicos

- Desarrollar una aplicación Android nativa utilizando Kotlin y Jetpack Compose.
- Diseñar una interfaz coherente con la identidad visual de Porto-Muiños y adaptable a diferentes configuraciones de dispositivos Android.
- Implementar un sistema de registro, autenticación, persistencia de sesión, cierre de sesión y recuperación de contraseña.
- Incorporar autenticación mediante Google OAuth.

- Utilizar Supabase como backend para almacenar y consultar productos, recetas y relaciones entre contenidos.
- Relacionar productos y recetas de forma bidireccional para facilitar la navegación contextual.
- Implementar mecanismos de búsqueda y filtrado sobre el catálogo y el recetario.
- Permitir al usuario gestionar productos y recetas favoritas asociadas a su cuenta.
- Implementar un carrito persistente por usuario y conectarlo con la tienda oficial de Porto-Muiños para completar la compra.
- Integrar un lector de códigos QR capaz de identificar productos y abrir directamente su ficha dentro de la aplicación.
- Desarrollar un demostrador de AlgaeChef capaz de generar recomendaciones controladas y enlazarlas con recetas reales disponibles en la aplicación.
- Aplicar una arquitectura MVVM apoyada en el patrón Repository para separar presentación, estado y acceso a datos.
- Proteger los datos privados mediante autenticación y políticas Row Level Security en Supabase.
- Validar el proyecto mediante pruebas unitarias, pruebas funcionales, inspección estática, emulador Android y dispositivo físico real.
- Documentar la arquitectura, el modelo de datos, las decisiones técnicas y el proceso de validación de forma comprensible y reproducible.

2.3. Grado de cumplimiento

Una vez finalizada la implementación, los objetivos definidos para el alcance del proyecto se consideran cumplidos. La tabla siguiente relaciona los objetivos principales con la evidencia disponible en la versión final del proyecto.

Objetivo	Resultado	Evidencia principal
Aplicación Android nativa	**Cumplido**	APK instalada y validada en emulador y dispositivo físico.
Backend remoto	**Cumplido**	Supabase con catálogo, relaciones, favoritos y carrito.
Autenticación	**Cumplido**	Correo, confirmación, recuperación y Google OAuth.
QR	**Cumplido**	Escaneo con cámara y resolución hacia fichas de producto.
Personalización	**Cumplido**	Favoritos y carrito persistentes por usuario.
Arquitectura y mantenibilidad	**Cumplido**	MVVM + Repository, separación por responsabilidades y auditoría final.
Seguridad	**Cumplido**	RLS por usuario y Edge Function autenticada para eliminación de cuentas.
Validación	**Cumplido**	13 pruebas unitarias, pruebas funcionales e inspección estática.
AlgaeChef	**Cumplido como demostrador**	Flujo conversacional, recomendaciones y acceso a recetas reales.

Tabla 2. Grado de cumplimiento de los objetivos principales.

3. Estudio de mercado y propuesta de valor

3.1. Público objetivo

La aplicación se orienta principalmente al consumidor final (B2C). La segmentación se ha definido a partir de situaciones de uso que encajan con las funcionalidades realmente implementadas. El objetivo no es limitar el producto a un único tipo de usuario, sino identificar necesidades concretas en las que una aplicación móvil puede reducir la incertidumbre sobre las algas y facilitar el paso desde el descubrimiento hasta la cocina o la compra.

Se distinguen cinco perfiles principales, todos ellos compatibles con el alcance actual de la aplicación:

Perfil	Necesidad principal	Funcionalidades de mayor valor
Consumidor curioso	Entender qué producto elegir y descubrir usos posibles.	Inicio, catálogo, recetas, fichas de producto y búsqueda.
Usuario principiante	Saber cómo utilizar un producto sin miedo a equivocarse.	Descripciones claras, recetas guiadas, productos relacionados y AlgaeChef.
Usuario práctico	Decidir qué cocinar con lo que tiene disponible o con una idea concreta.	Buscador, recetario, favoritos y demostrador AlgaeChef.
Cliente recurrente	Volver a localizar y comprar productos con rapidez.	Cuenta de usuario, favoritos, carrito persistente y acceso a la tienda oficial.
Usuario con envase físico	Acceder a información útil sin realizar una búsqueda manual.	Escáner QR, ficha del producto, recetas relacionadas y acceso a compra.

Tabla 3. Perfiles de usuario considerados en el proyecto.

Estos perfiles no representan tipos de cuenta ni roles técnicos diferentes dentro de la aplicación; sirven únicamente para describir necesidades y contextos de uso distintos sobre una misma experiencia B2C.

3.2. Estado actual y soluciones existentes

El punto de partida digital de Porto-Muiños ya es relevante: su web oficial reúne información sobre algas, productos y recetas, y dispone además de una tienda en línea desde la que se pueden consultar referencias, precios y realizar compras [1]. Por tanto, la oportunidad del proyecto no consiste en sustituir esa infraestructura, sino en organizar parte de ese conocimiento dentro de una experiencia móvil más contextual.

En paralelo, el mercado de aplicaciones culinarias muestra una tendencia consolidada hacia la combinación de varias funciones en un mismo entorno. SideChef integra descubrimiento de recetas, planificación y compra de ingredientes, incluyendo recetas comprables conectadas con distribuidores [2]. Samsung Food combina guardado de recetas, planificación de comidas, listas de compra y personalización alimentaria [3]. Kitchen Stories centra su propuesta en el descubrimiento de recetas, filtros por tipo de plato, dieta, tiempo de preparación e ingredientes principales [4].

Estas soluciones demuestran que existe un patrón de uso ya asentado: el usuario valora que la inspiración culinaria no termine en una receta aislada, sino que pueda continuar hacia la planificación, la compra o la personalización. Sin embargo, se trata de plataformas generalistas. La propuesta de Porto-Muiños App se sitúa en un nicho diferente: una aplicación vinculada a una marca y a una categoría gastronómica concreta, las algas y verduras del mar.

3.3. Competidores y alternativas

No se ha identificado una aplicación móvil oficial de Porto-Muiños equivalente a la solución desarrollada. La comparación se realiza por tanto con la web de la propia empresa y con aplicaciones culinarias que compiten de forma indirecta por cubrir necesidades similares.

Solución	Fortalezas principales	Diferencia respecto a Porto-Muiños App
Web oficial de Porto-Muiños [1]	Catálogo real de la marca, recetas, información corporativa y tienda online.	La app añade navegación móvil nativa, cuenta de usuario, favoritos, carrito propio, QR y conexión contextual entre producto y receta.
SideChef [2]	Recetas paso a paso, planificación y compra directa de ingredientes.	Es generalista y orientada a múltiples marcas y distribuidores; no está especializada en algas ni conecta envases Porto-Muiños mediante QR.
Samsung Food [3]	Recetario, planificación, listas de compra, comunidades y personalización.	Su alcance es mucho más amplio, pero no actúa como extensión digital de una marca alimentaria concreta.
Kitchen Stories [4]	Contenido editorial, recetas visuales y filtros culinarios.	Está centrada en inspiración y cocina general; no integra el catálogo comercial ni el vínculo físico-digital planteado en este proyecto.

Tabla 4. Comparación cualitativa con soluciones existentes.

La conclusión del análisis es que la competencia directa es limitada, mientras que la competencia funcional es amplia. Porto-Muiños App no pretende competir con plataformas generalistas por volumen de recetas; su valor está en especializar la experiencia alrededor del catálogo, el conocimiento culinario y los canales comerciales de Porto-Muiños.

3.4. Diferenciación de Porto-Muiños App

La propuesta de valor se basa en convertir contenidos que habitualmente aparecen separados —producto, receta, recomendación y compra— en un flujo continuo. El usuario puede descubrir una receta, consultar los productos Porto-Muiños asociados, acceder a la ficha de cada producto y continuar hacia la tienda oficial. El recorrido también funciona en sentido inverso: desde un producto pueden consultarse recetas relacionadas.

A esta relación se añaden dos elementos diferenciales. El primero es el escáner QR, que conecta un envase físico con su ficha digital dentro de la aplicación. El segundo es AlgaeChef, implementado en esta versión como demostrador de un futuro asistente culinario capaz de orientar al usuario y devolver recomendaciones vinculadas con contenido real del catálogo.

La aplicación incorpora además estado asociado a cada cuenta —favoritos y carrito persistentes— y mantiene el proceso de compra conectado con la infraestructura comercial existente en lugar de duplicar un checkout completo. De esta forma se prioriza la utilidad móvil sin reemplazar innecesariamente sistemas que Porto-Muiños ya dispone en la web.

3.5. Viabilidad de la propuesta

Desde el punto de vista de mercado, la propuesta se considera viable como canal digital complementario de la marca. Las soluciones analizadas muestran que combinar recetas con planificación o compra es un modelo de interacción habitual [2][3], mientras que Porto-Muiños ya dispone de contenido gastronómico, catálogo y comercio electrónico propios [1]. Esto reduce la necesidad de construir desde cero el contenido y la infraestructura comercial.

La especialización también limita la competencia directa: la aplicación no necesita alcanzar el volumen de contenido de una plataforma generalista, sino resolver mejor un problema concreto, ayudar a comprender y utilizar productos de algas. El QR, la relación producto-receta y el acceso contextual a compra refuerzan esa orientación.

No obstante, esta viabilidad debe interpretarse de forma cualitativa. El proyecto académico valida la factibilidad técnica y la experiencia de uso, pero no dispone de métricas reales de lanzamiento, retención, conversión o incremento de ventas. Una implantación comercial requeriría medir esos indicadores y mantener actualizado el catálogo, los enlaces de compra y el contenido editorial. Por ello, la aplicación se plantea como una base funcional y escalable cuya rentabilidad empresarial debería validarse posteriormente con datos reales.

4. Análisis y requisitos

4.1. Requisitos funcionales

Los requisitos funcionales describen las capacidades que la aplicación debe ofrecer al usuario. Se han definido a partir del alcance realmente implementado y validado en la versión final, evitando incluir como requisito aquello que permanece únicamente como mejora futura.

ID	Requisito funcional	Estado
RF-01	Permitir el registro mediante correo electrónico y contraseña.	Implementado
RF-02	Permitir el inicio de sesión mediante correo electrónico y contraseña.	Implementado
RF-03	Permitir el acceso mediante Google OAuth.	Implementado
RF-04	Permitir recuperar la contraseña mediante correo y deep link.	Implementado
RF-05	Mantener la sesión, cerrar sesión y eliminar la cuenta de usuario.	Implementado
RF-06	Consultar el catálogo de productos almacenado en Supabase.	Implementado
RF-07	Consultar recetas con ingredientes, pasos y metadatos básicos.	Implementado
RF-08	Buscar y filtrar productos y recetas.	Implementado
RF-09	Navegar entre productos y recetas relacionados.	Implementado
RF-10	Añadir y eliminar productos y recetas de favoritos por usuario.	Implementado
RF-11	Gestionar un carrito persistente con cantidades por producto.	Implementado
RF-12	Continuar la compra en la tienda oficial de Porto-Muiños.	Implementado
RF-13	Escanear códigos QR y resolverlos hacia la ficha interna del producto.	Implementado
RF-14	Consultar AlgaeChef y recibir una recomendación controlada vinculada a una receta real.	Implementado como demostrador
RF-15	Mostrar el perfil del usuario y permitir seleccionar una imagen de perfil local.	Implementado

Tabla 5. Requisitos funcionales de la versión final.

4.2. Requisitos no funcionales

Los requisitos no funcionales establecen las condiciones de calidad que debe cumplir la solución. En un proyecto móvil resultan especialmente relevantes la usabilidad, la seguridad, la mantenibilidad y la adaptación a diferentes configuraciones del sistema.

ID	Requisito no funcional	Estado
RNF-01 · Usabilidad	La navegación debe ser comprensible, consistente y mantener visibles las acciones principales.	Cumplido
RNF-02 · Adaptabilidad	La interfaz debe respetar barras del sistema e insets tanto con navegación gestual como con navegación clásica de tres botones.	Cumplido
RNF-03 · Seguridad	Los datos privados de favoritos y carrito deben aislarse por usuario mediante autenticación y políticas RLS.	Cumplido
RNF-04 · Persistencia	Sesión, favoritos y carrito deben mantenerse correctamente entre aperturas de la aplicación y entre dispositivos cuando proceda.	Cumplido
RNF-05 · Mantenibilidad	La lógica debe separarse de la interfaz mediante MVVM, repositorios y componentes reutilizables.	Cumplido
RNF-06 · Legibilidad	El código debe emplear nomenclatura homogénea, responsabilidades claras y comentarios breves donde aporten contexto.	Cumplido
RNF-07 · Compatibilidad	La aplicación debe funcionar en versiones Android compatibles con la configuración del proyecto y en dispositivo físico.	Cumplido
RNF-08 · Privacidad	Las credenciales de Supabase no deben quedar codificadas directamente en el repositorio; se cargan desde configuración local.	Cumplido
RNF-09 · Robustez	Los fallos de autenticación, red o servicios externos deben traducirse a mensajes comprensibles y no provocar cierres inesperados.	Cumplido
RNF-10 · Reproducibilidad	El proyecto debe incluir una plantilla de configuración, el esquema SQL y la Edge Function necesarios para reproducir el backend.	Cumplido

Tabla 6. Requisitos no funcionales principales.

4.3. Actores del sistema

El sistema distingue actores humanos y servicios externos. Aunque el usuario es el actor principal, determinados casos de uso dependen de proveedores externos que forman parte del flujo completo.

Actor	Responsabilidad o interacción
Usuario no autenticado	Registrarse, iniciar sesión, solicitar recuperación de contraseña o continuar con Google.
Usuario autenticado	Consultar contenido, buscar, escanear QR, gestionar favoritos y carrito, utilizar AlgaeChef y administrar su sesión.
Google OAuth	Validar la identidad del usuario cuando se selecciona el acceso con Google.
Supabase	Proporcionar autenticación, base de datos, almacenamiento, APIs y ejecución de la Edge Function.
Tienda oficial de Porto-Muiños	Completar fuera de la aplicación el proceso real de compra y pago.

Tabla 7. Actores y sistemas externos implicados.

4.4. Casos de uso

Los casos de uso sintetizan los recorridos principales del usuario y reflejan únicamente comportamientos presentes en la aplicación final.

ID	Caso de uso	Resultado esperado
CU-01	Registrarse o iniciar sesión	El usuario obtiene una sesión válida y accede al contenido principal.
CU-02	Recuperar contraseña	El usuario recibe el enlace de recuperación y establece una nueva contraseña.
CU-03	Consultar producto	Visualiza imagen, descripción, precio, relaciones y acciones disponibles.
CU-04	Consultar receta	Visualiza descripción, ingredientes, pasos y productos asociados.
CU-05	Buscar y filtrar	Reduce el catálogo o el recetario según texto y categoría.
CU-06	Gestionar favoritos	Añade o elimina productos y recetas y conserva el estado en su cuenta.
CU-07	Gestionar carrito	Añade productos, modifica cantidades, elimina líneas o vacía el carrito.
CU-08	Escanear QR	La cámara identifica un código compatible y abre la ficha interna del producto.
CU-09	Consultar AlgaeChef	Recibe una recomendación controlada y una receta asociada a partir de su consulta.
CU-10	Continuar compra	La aplicación redirige al usuario a la ficha oficial o a la tienda de Porto-Muiños.
CU-11	Gestionar sesión	Consulta su perfil, cierra sesión o elimina definitivamente la cuenta.

Tabla 8. Casos de uso principales.

4.5. Diagrama de casos de uso

La Figura 1 representa los principales casos de uso distinguiendo entre usuario no autenticado y usuario autenticado. Google OAuth y la tienda oficial se muestran como sistemas externos porque intervienen respectivamente en el acceso mediante cuenta de Google y en la finalización del proceso real de compra.

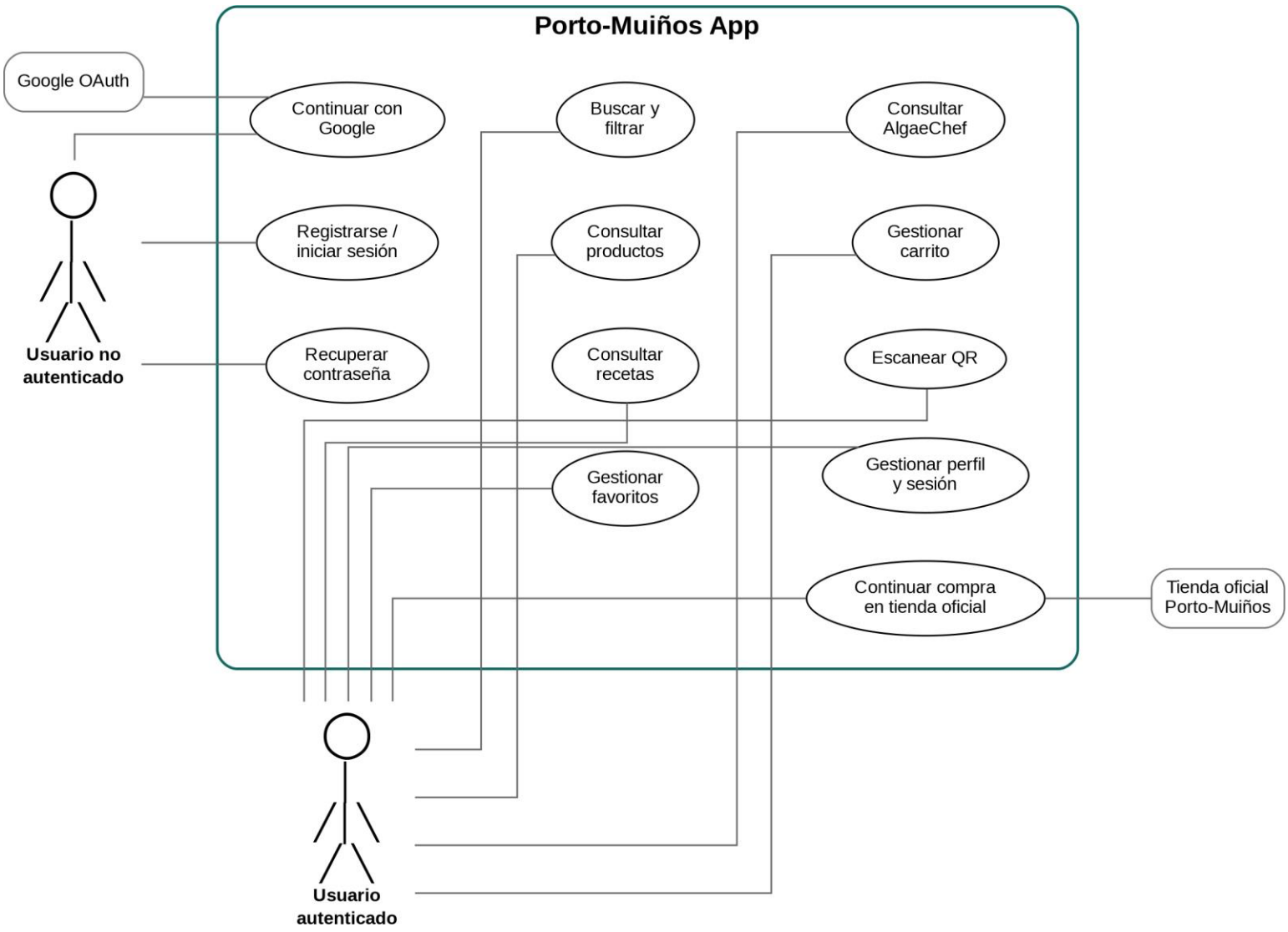


Figura 1. Diagrama de casos de uso de Porto-Muiños App.

Este diagrama identifica quién puede realizar cada acción y qué servicios externos intervienen; no representa el orden temporal de las pantallas. El recorrido completo de navegación se documenta de forma separada en el apartado 7.3.

5. Planificación y metodología de desarrollo

5.1. Fases del proyecto

El desarrollo se ha realizado de forma incremental. No se aplicó una metodología ágil formal con sprints cerrados, sino una planificación práctica por bloques funcionales,

priorizando primero aquellas partes que condicionaban el resto de la solución. Esta decisión permitió validar cada avance antes de incorporar nuevas responsabilidades.

Fase	Periodo aproximado	Contenido principal
1. Conceptualización previa	Enero-mayo de 2026	Reto de InnovaTech, definición de la propuesta, perfiles de usuario, mockups y alcance inicial.
2. Redefinición individual	Agosto de 2026	Adaptación del concepto al Proyecto de Fin de Ciclo, elección de Kotlin/Compose y delimitación del alcance real.
3. Base funcional y datos	Primera mitad de agosto	Estructura Android, catálogo remoto, recetas, imágenes, navegación y conexión con Supabase.
4. Funcionalidades de usuario	Mitad de agosto	Autenticación, recuperación de contraseña, Google OAuth, QR, buscadores, favoritos y carrito.
5. Cierre funcional	Segunda mitad de agosto	Integración con tienda oficial, detalle de producto/receta, perfil, AlgaeChef demostrador y pruebas completas.
6. Refactorización y auditoría	Segunda mitad de agosto	Migración a MVVM, separación de responsabilidades, tests, revisión RLS, backend reproducible e inspección estática.
7. Validación y documentación	Parte final de agosto	Pruebas en móvil físico, ajustes de insets, icono final, memoria, APK y preparación de la entrega.

Tabla 9. Fases principales del desarrollo.

5.2. Priorización funcional

La priorización se basó en dos criterios: valor para el flujo principal y riesgo técnico. Primero se cerraron las piezas que daban estructura a la aplicación y después las integraciones más complejas. De esta forma, una funcionalidad avanzada no podía comprometer la estabilidad del conjunto.

Prioridad	Funcionalidades	Criterio
Esencial	Autenticación, catálogo, recetas, navegación, Supabase y relación producto-receta.	Constituyen el núcleo mínimo de la aplicación.
Alta	QR, búsqueda, favoritos, carrito, persistencia e integración con tienda.	Aportan el valor diferencial y requieren integración técnica real.
Complementaria	Google OAuth, recuperación de contraseña, perfil y AlgaeChef demostrador.	Mejoran la experiencia sin bloquear el flujo principal.
Futura	IA generativa real, Apple, preferencias dietéticas, alergias y proceso de compra integrado en la app (checkout propio).	Su coste, complejidad o dependencia externa no compensaba el riesgo en la fase final; la compra actual se completa en la tienda oficial.

Tabla 10. Priorización funcional aplicada.

5.3. Evolución del proyecto

El proyecto evolucionó de forma significativa desde la propuesta de InnovaTech hasta la aplicación final. En la memoria inicial se planteaba un desarrollo multiplataforma con React Native y un AlgaeChef basado en inteligencia artificial. Durante la redefinición individual se optó por Android nativo con Kotlin y Jetpack Compose, manteniendo Supabase como backend. Esta decisión redujo la dispersión tecnológica y facilitó una integración más directa con cámara, navegación y componentes Android.

También se modificó el planteamiento de compra. En lugar de replicar dentro de la aplicación un sistema completo de checkout, el carrito sirve como herramienta de selección y el proceso de pago se completa en la tienda oficial. De este modo se evita duplicar una infraestructura comercial existente y se mantiene el proyecto dentro de un alcance asumible.

AlgaeChef siguió una evolución similar. Se conservó la experiencia de conversación, las opciones rápidas y la recomendación de recetas, pero se implementó mediante lógica controlada. La integración de un modelo generativo se documenta como mejora futura porque no era necesaria para demostrar el flujo de producto ni justificaba introducir una dependencia adicional en la fase de cierre.

Finalmente, una vez estabilizada la funcionalidad, la aplicación se refactorizó a una arquitectura MVVM con repositorios. La refactorización se realizó al final para no mezclar cambios estructurales con la implementación de nuevas funciones y permitió mejorar la mantenibilidad sin alterar el comportamiento validado.

5.4. Gestión de riesgos y alcance

La gestión del alcance fue determinante. La propuesta conceptual era mayor que el tiempo disponible para el proyecto, por lo que se identificaron riesgos técnicos y se adoptaron decisiones concretas para reducir su impacto.

Riesgo	Impacto potencial	Medida adoptada
Alcance excesivo	No finalizar una versión estable.	Congelar funcionalidades no esenciales y trasladarlas a líneas futuras.
Autenticación y deep links	Bloqueo del acceso o recuperación de contraseña.	Implementación incremental y pruebas repetidas con correo y Google OAuth.
Variabilidad de códigos QR	Códigos reales con formatos diferentes.	Resolver identificadores internos, URLs y mapeos hacia la misma ficha de producto.
Persistencia multiusuario	Datos de favoritos o carrito mezclados entre cuentas.	Tablas asociadas a auth.users y políticas RLS por user_id.
Dependencia de la tienda externa	Enlaces que cambian o páginas no comprables.	Guardar source_url por producto y validar manualmente cada enlace.
Interfaz en distintos dispositivos	Solapamientos con barras del sistema.	Uso de WindowInsets y pruebas con gestos y navegación de tres botones.
Complejidad de IA generativa	Tiempo, costes y comportamiento no determinista.	Mantener AlgaeChef como demostrador controlado y documentar la IA real como ampliación.

Tabla 11. Riesgos principales y medidas de mitigación.

6. Presupuesto y viabilidad económica

6.1. Recursos humanos

Los recursos humanos del desarrollo presentado están constituidos por una única persona: Miguel Fernández González, alumno de 2º DAM Dual y responsable individual del análisis, diseño, implementación Android, configuración del backend en Supabase, integraciones, pruebas, refactorización y documentación. Porto-Muiños actúa como entidad de referencia para la finalidad, la identidad visual y los contenidos del producto, pero no forma parte del equipo de desarrollo de esta versión.

Para cuantificar económicamente el trabajo se utiliza una estimación retrospectiva de horas por áreas. No se realizó un parte horario formal durante el desarrollo, por lo que las cifras no se presentan como facturación real, sino como una aproximación al coste que tendría ejecutar un proyecto equivalente en un contexto profesional.

6.2. Coste estimado del desarrollo

Se toma como referencia un coste profesional conservador de 20 €/hora. La cifra se utiliza únicamente para traducir el esfuerzo realizado a una valoración económica comprensible y no representa el salario efectivo ni un presupuesto comercial emitido por una empresa.

Área	Horas estimadas	Coste a 20 €/h
Análisis y requisitos	20 h	400 €
Diseño UX/UI y adaptación visual	35 h	700 €
Desarrollo Android y navegación	80 h	1.600 €
Backend Supabase y modelo de datos	35 h	700 €
Integraciones: OAuth, QR y tienda	20 h	400 €
Pruebas, refactorización y auditoría	25 h	500 €
Documentación y preparación de entrega	15 h	300 €
TOTAL	230 h	4.600 €

Tabla 12. Estimación del coste profesional equivalente por áreas.

6.3. Herramientas y servicios

La mayor parte de las herramientas utilizadas disponen de versiones gratuitas suficientes para el desarrollo académico. Esto reduce prácticamente a cero el desembolso directo incremental del proyecto, ya que también se ha utilizado equipamiento previamente disponible.

Recurso	Uso en el proyecto	Coste directo en el desarrollo
Android Studio, Kotlin y Jetpack Compose	Desarrollo de la aplicación Android.	0 €
JDK y Gradle	Compilación, gestión de dependencias y tests.	0 €
Supabase Free	Base de datos, Auth, Storage, APIs y Edge Function.	0 €
Figma / herramientas de prototipado	Diseño y validación de interfaces.	0 € en el alcance utilizado
Emulador Android	Pruebas sobre distintas configuraciones.	0 €
PC y teléfono de pruebas	Desarrollo y validación en dispositivo real.	Equipamiento ya disponible
Conexión a Internet	Acceso a servicios cloud, documentación y pruebas.	Coste general no imputado de forma independiente
ChatGPT	Herramienta de apoyo para contraste de decisiones, depuración, revisión técnica y elaboración de documentación. No forma parte de la ejecución de la aplicación.	Herramienta personal de apoyo; coste no imputado al presupuesto del proyecto

Tabla 13. Herramientas, servicios y desembolso directo.

6.4. Infraestructura

La infraestructura actual está dimensionada para un entorno académico y demostrativo. Supabase concentra autenticación, PostgreSQL, Storage y funciones serverless, evitando administrar un servidor propio. La aplicación consume estos servicios mediante las librerías integradas y completa la compra mediante enlaces a la infraestructura web existente de Porto-Muiños.

Para una puesta en producción comercial no sería metodológicamente correcto extrapolar el coste actual sin conocer el número de usuarios, el volumen de almacenamiento, el tráfico, la frecuencia de uso o las necesidades de soporte. En lugar de asignar importes ficticios, se identifican los elementos que deberían presupuestarse y validarse mediante un piloto antes del lanzamiento.

Elemento	Situación académica	Consideración para producción
Backend y base de datos	Plan de Supabase suficiente para pruebas y demostración.	Revisar límites de usuarios, almacenamiento, transferencia, funciones y copias de seguridad.
Correo transaccional	Servicio integrado suficiente para pruebas, con límites de envío.	Configurar un proveedor SMTP propio y dimensionar el volumen de confirmaciones y recuperaciones.
Publicación	APK instalada directamente en dispositivos de prueba.	Preparar versión release, firma, ficha de tienda y costes asociados a la distribución.
Monitorización y mantenimiento	Revisión manual durante el desarrollo.	Definir seguimiento de errores, actualizaciones, compatibilidad y mantenimiento periódico.
Contenido y enlaces comerciales	Catálogo y enlaces validados manualmente.	Establecer un proceso de actualización de productos, recetas, imágenes y URLs de compra.

Tabla 14. Aspectos de infraestructura a revisar antes de una puesta en producción comercial.

6.5. Presupuesto total estimado

A partir de la valoración anterior se distinguen dos magnitudes: el desembolso directo realizado para el proyecto académico y el valor profesional equivalente del desarrollo. Esta separación evita presentar como gasto real recursos que ya estaban disponibles y, al mismo tiempo, permite estimar de forma prudente el esfuerzo técnico realizado.

Concepto	Importe
Coste directo incremental del proyecto académico	0 € aprox.
Valor profesional estimado del desarrollo (230 h × 20 €/h)	4.600 €
Presupuesto profesional equivalente estimado	4.600 €

Tabla 15. Resumen del presupuesto estimado.

La estimación no incluye marketing, acciones promocionales, publicación comercial ni mantenimiento posterior. Tampoco se proyectan ingresos o incrementos de ventas, ya que no se dispone de datos reales de Porto-Muiños sobre adopción, conversión, ticket medio o recompra que permitan justificar una previsión financiera fiable. En una implantación empresarial, estas métricas deberían validarse mediante un lanzamiento piloto y utilizarse después para dimensionar la infraestructura y evaluar el retorno de la inversión.

7. Diseño de la solución

7.1. Diseño de interfaz y experiencia de usuario

El diseño de Porto-Muiños App se planteó con un criterio mobile-first y con una prioridad clara: reducir la carga cognitiva de un usuario que puede no estar familiarizado con las algas. La interfaz evita presentar información técnica de forma aislada y organiza el contenido alrededor de acciones comprensibles: descubrir, buscar, consultar una receta, conocer un producto, guardarlo, añadirlo al carrito o acceder a una recomendación de AlgaeChef.

Las pantallas se construyen con Jetpack Compose y mantienen patrones comunes de interacción. La barra de navegación inferior da acceso persistente a Inicio, Recetas, AlgaeChef y Tienda, mientras que el escáner QR ocupa una posición central destacada. Las fichas de producto y receta utilizan jerarquía tipográfica, imágenes de catálogo, descripciones breves y acciones contextuales para favorecer una lectura rápida.

La experiencia se ha validado tanto en emulador como en dispositivo físico. Esta validación permitió detectar ajustes que no resultaban evidentes en una única configuración, como el espacio ocupado por la navegación clásica de tres botones de Android, y adaptar los márgenes mediante WindowInsets sin modificar individualmente cada pantalla.

7.2. Sistema visual

La identidad visual toma como referencia directa la imagen de Porto-Muiños y se adapta a una interfaz móvil limpia y reconocible. La paleta combina el verde principal y el verde oscuro con tonos menta y superficies claras; Poppins se utiliza como familia tipográfica en sus variantes Regular, Medium y SemiBold. La Figura 2 completa la tabla cromática con recursos reales del proyecto: logotipo, mascota AlgaeChef, botones, campos de formulario, buscador y filtros, una tarjeta de receta y el patrón de navegación inferior con el botón QR central.

Elemento	Aplicación en la interfaz
Verde principal · #028174	Acciones destacadas, selección activa y elementos interactivos.
Verde oscuro · #0E3D3A	Texto de énfasis, iconografía y contraste de marca.
Menta · #A8D5C2	Fondos secundarios, estados seleccionados y apoyo visual.
Fondos claros	Superficies #FAFBFB y #F2F6F5 para mantener legibilidad y ligereza.
Poppins	Familia tipográfica de la app: Regular, Medium y SemiBold.
Iconografía	Vectores simples y consistentes; icono launcher basado en el símbolo de Porto-Muiños.

Tabla 16. Principales decisiones del sistema visual de Porto-Muiños App.

Sistema visual de Porto-Muiños App

Paleta, tipografía, componentes e identidad visual utilizados en la aplicación

Paleta principal

Verde principal	Verde oscuro	Menta	Superficie	Fondo
#028174	#0E3D3A	#A8D5C2	#F2F6F5	#FAFBFB

Tipografía · Poppins

Titular SemiBold

Subtítulo Medium

Texto Regular para contenido, descripciones y ayuda.

Botones

Acción principal

Acción secundaria

Identidad visual

PORTO·MUIÑOS
LAS VERDURAS DEL MAR



AlgaeChef

Componentes reales de la interfaz

Búsqueda y filtros



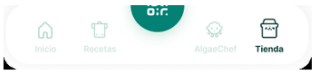
Campos de formulario



Tarjeta de receta



Navegación inferior



La barra inferior, el botón QR central y los estados seleccionados forman un patrón común en las pantallas principales.

Figura 2. Sistema visual de Porto-Muiños App a partir de recursos y componentes utilizados en la interfaz.

7.3. Flujo de navegación

El flujo de navegación complementa el diagrama de casos de uso del capítulo 4. Mientras aquel identifica actores y capacidades, el siguiente esquema representa el orden y las principales transiciones entre pantallas. La aplicación parte de Splash, comprueba la sesión y conduce al usuario al acceso o a Home. Desde la navegación principal se abren los recorridos de recetas, tienda, QR, AlgaeChef y perfil.

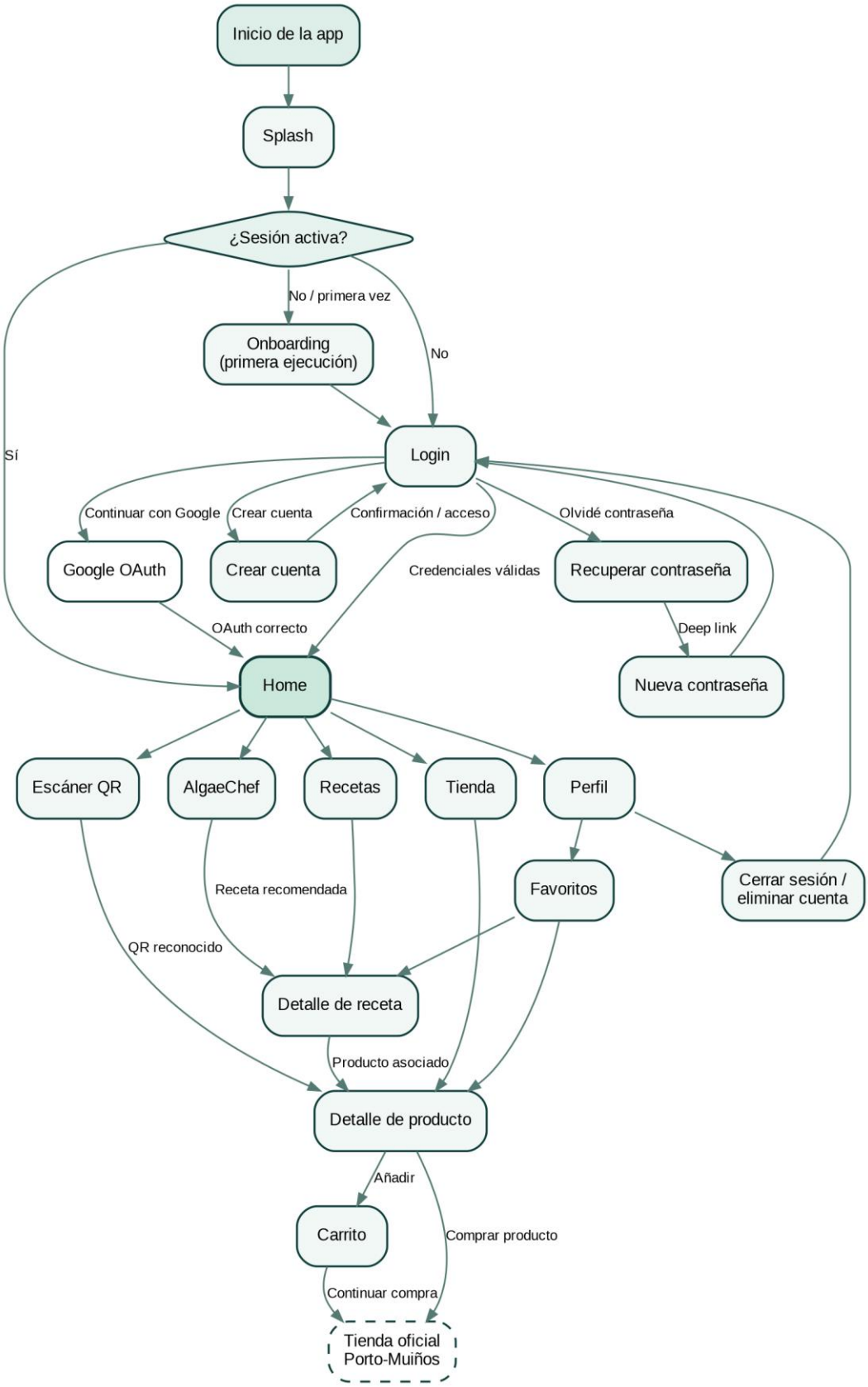


Figura 3. Flujo principal de navegación de Porto-Muiños App.

Los recorridos no son compartimentos aislados. El detalle de una receta puede conducir a productos asociados; el detalle de un producto puede abrir recetas relacionadas o el carrito; un QR reconocido desemboca en la ficha interna del producto; y una recomendación de AlgaeChef enlaza con una receta existente. Esta interconexión constituye uno de los criterios de diseño centrales del proyecto.

7.4. Wireframes y prototipos

Figma se utilizó para consolidar un prototipo visual avanzado antes de cerrar la implementación. En esta fase ya estaban definidos el lenguaje gráfico principal, la estructura de navegación, las pantallas de autenticación, Inicio, Recetas, Tienda, AlgaeChef y el flujo de escaneo QR.

La composición de la Figura 4 corresponde a una fase avanzada y próxima al resultado final, aunque todavía contiene decisiones que posteriormente se ajustaron durante el desarrollo. La implementación real incorporó cambios en textos de autenticación, persistencia por usuario, carrito, favoritos, adaptación a insets de Android y otros detalles de interacción detectados durante las pruebas.



Figura 4. Conjunto de mockups del prototipo visual avanzado previo a la implementación final.

7.5. Adaptación a diferentes configuraciones Android

La interfaz se diseñó para funcionar desde Android 8.0 (API 26) y se compiló con SDK 36. Además de comprobar diferentes tamaños mediante el emulador, se realizaron pruebas en un teléfono físico. Una incidencia relevante apareció al utilizar navegación de tres botones: la barra inferior del sistema ocupa más altura que la navegación por gestos y podía acercar o solapar contenido situado al final de algunas pantallas.

La solución se centralizó en los componentes compartidos de navegación utilizando los insets reales del sistema. De esta forma, el espacio de seguridad de la barra inferior se calcula en función de WindowInsets.navigationBars y se adapta automáticamente a navegación gestual o clásica. Los mensajes transitorios emplean el mismo principio cuando se muestran en pantallas sin la barra de navegación propia de la app.

8. Arquitectura y diseño técnico

8.1. Arquitectura general del sistema

La versión final adopta una arquitectura MVVM apoyada por el patrón Repository. El objetivo es evitar que las pantallas Compose conozcan directamente los detalles de Supabase y separar tres responsabilidades: representación visual, estado/lógica de presentación y acceso a datos. La navegación se mantiene como una responsabilidad diferenciada mediante Navigation Compose.

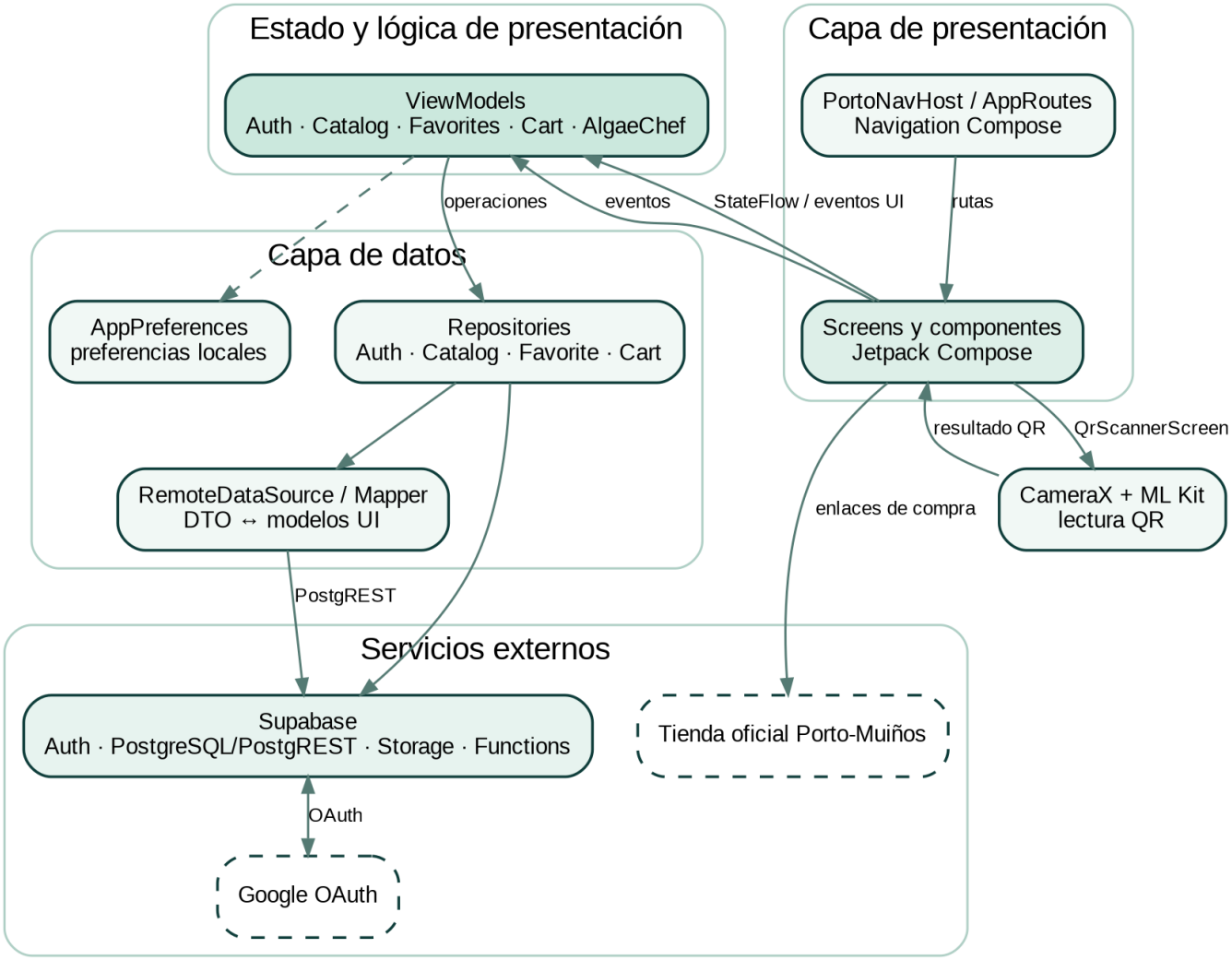


Figura 5. Arquitectura general MVVM + Repository utilizada en la aplicación.

La aplicación no incorpora una capa Domain o un sistema de inyección de dependencias porque, para el alcance actual, añadir esas capas aumentaría la complejidad sin resolver un problema real. La estructura elegida mantiene las responsabilidades suficientemente aisladas y permite explicar con claridad el recorrido de una operación desde la interfaz hasta el backend.

8.2. Patrón MVVM

MVVM (Model-View-ViewModel) es un patrón arquitectónico que separa la representación de la interfaz, el estado y la lógica de presentación del acceso al modelo

de datos. Su finalidad es reducir el acoplamiento entre pantallas y operaciones, facilitar el mantenimiento y permitir que el estado evolucione de forma independiente a la composición visual. En Porto-Muiños App, las Screens representan la View; los ViewModels exponen el estado y reciben las acciones del usuario; y los repositories/modelos proporcionan los datos necesarios. StateFlow, un flujo observable de Kotlin que conserva y emite el valor de estado más reciente, permite que Compose reaccione a sus cambios mediante recomposición automática. [5][6]

ViewModel	Responsabilidad principal
AuthViewModel	Registro, login, sesión, recuperación de contraseña, Google OAuth y acciones de cuenta.
CatalogViewModel	Carga y mantenimiento del catálogo remoto compartido por las pantallas.
FavoritesViewModel	Favoritos de productos y recetas, actualización optimista y sincronización persistente.
CartViewModel	Estado del carrito, cantidades, actualización optimista y sincronización ordenada con Supabase.
AlgaeChefViewModel	Estado conversacional del demostrador y generación controlada de recomendaciones.

Tabla 17. ViewModels y responsabilidades de la versión final.

8.3. Patrón Repository

El patrón Repository introduce una capa que abstrae el origen y la forma de acceso a los datos. La interfaz y los ViewModels solicitan operaciones al repository sin necesitar conocer si la información procede de una API, una base de datos remota o almacenamiento local. En Porto-Muiños App, AuthRepository concentra las operaciones de Supabase Auth; CatalogRepository coordina el catálogo; FavoriteRepository gestiona altas y bajas de favoritos; y CartRepository encapsula las operaciones del carrito. Esta separación evita repetir consultas PostgREST o detalles de autenticación en las pantallas y permite modificar la fuente de datos con un impacto menor sobre la UI. [5]

CatalogRemoteDataSource representa además una separación adicional para la lectura del catálogo remoto. Los DTO recibidos desde Supabase se transforman mediante CatalogMapper a modelos preparados para la interfaz, incluyendo la construcción de URLs públicas del bucket de imágenes.

8.4. Separación de responsabilidades

La auditoría final del proyecto se centró en que cada archivo tuviese una responsabilidad reconocible. Las Screens se ocupan de disposición y eventos; los componentes reutilizables contienen patrones visuales; los ViewModels coordinan estado y operaciones; los repositories acceden a Supabase; los DTO representan respuestas remotas; y los modelos UI contienen la información que consume Compose. Esta estructura reduce dependencias cruzadas y facilita localizar el origen de un comportamiento durante el mantenimiento, la depuración y la evolución del proyecto.

8.5. Gestión del estado

Los ViewModels utilizan MutableStateFlow internamente y exponen StateFlow de solo lectura a la interfaz. StateFlow representa un estado observable y conserva siempre su último valor, por lo que resulta apropiado para catálogo cargado, favoritos, carrito o estados de autenticación. Para eventos puntuales que no deben permanecer como estado —por ejemplo, un error o una confirmación mostrada una sola vez— se utiliza SharedFlow, que distribuye emisiones a los observadores sin exigir conservar un valor permanente. Los mensajes se encapsulan mediante el modelo común UiMessage. Esta separación evita que una notificación temporal reaparezca al recomponer una pantalla.

Favoritos y carrito aplican actualización optimista: la interfaz refleja inmediatamente la acción del usuario y posteriormente sincroniza el cambio con Supabase. Si la escritura falla, el ViewModel puede restaurar el estado o recargarlo desde servidor y mostrar un aviso. En operaciones rápidas sobre un mismo elemento, las escrituras se encadenan para conservar el orden y evitar inconsistencias.

8.6. Navegación

Navigation Compose es la librería de Jetpack que permite declarar y gestionar destinos y transiciones entre pantallas construidas con Compose. En el proyecto gestiona el grafo de navegación: AppRoutes centraliza las rutas literales y define argumentos para los detalles de producto y receta; PortoNavHost declara los destinos y conecta los ViewModels con cada Screen; y PortoMuinosApp actúa como punto de coordinación raíz. Los deep links de autenticación se procesan fuera de las Screens y permiten distinguir la recuperación de contraseña del retorno OAuth. [8]

8.7. Estructura de paquetes y carpetas

La estructura de paquetes refleja las responsabilidades arquitectónicas y evita concentrar todo el proyecto en un único directorio. La organización principal es la siguiente:

- data/auth: gestión de deep links de autenticación.
- data/local: preferencias locales de la aplicación.
- data/mapper, data/remote y data/repository: transformación, fuentes remotas y repositories.
- ui/screens: pantallas Compose completas.
- ui/components: componentes visuales reutilizables.
- ui/viewmodel: estado y lógica de presentación.
- ui/navigation: rutas, grafo y barra de navegación.
- ui/model, ui/search, ui/qr y ui/theme: modelos de interfaz, búsqueda, resolución QR y sistema visual.

8.8. Componentes reutilizables

La reutilización se aplica a elementos que aparecen en varias pantallas o que encapsulan un comportamiento visual concreto. Entre ellos se encuentran PortoBottomBar, PortoBackButton, PortoSearchBar, CatalogImage, FavoriteIconButton, CartIconButton, ProfileAvatar, RecipeCard, FeaturedRecipeCard, FfeaturedProductCard y PortoTransientMessageHost. Centralizar estos elementos reduce duplicación y permite resolver cambios transversales —por ejemplo, los insets de navegación— en un único punto.

9. Modelo de datos y backend

9.1. Arquitectura de Supabase

Supabase funciona como Backend as a Service y concentra los servicios remotos de la aplicación. PostgreSQL almacena catálogo y datos por usuario; PostgREST expone las tablas mediante API; Auth gestiona identidad y sesiones; Storage publica las imágenes del catálogo; y Functions aloja la función serverless empleada para eliminar cuentas. El

cliente Android se configura desde SupabaseClientProvider e instala únicamente los módulos necesarios: Postgrest, Auth y Functions. [11]

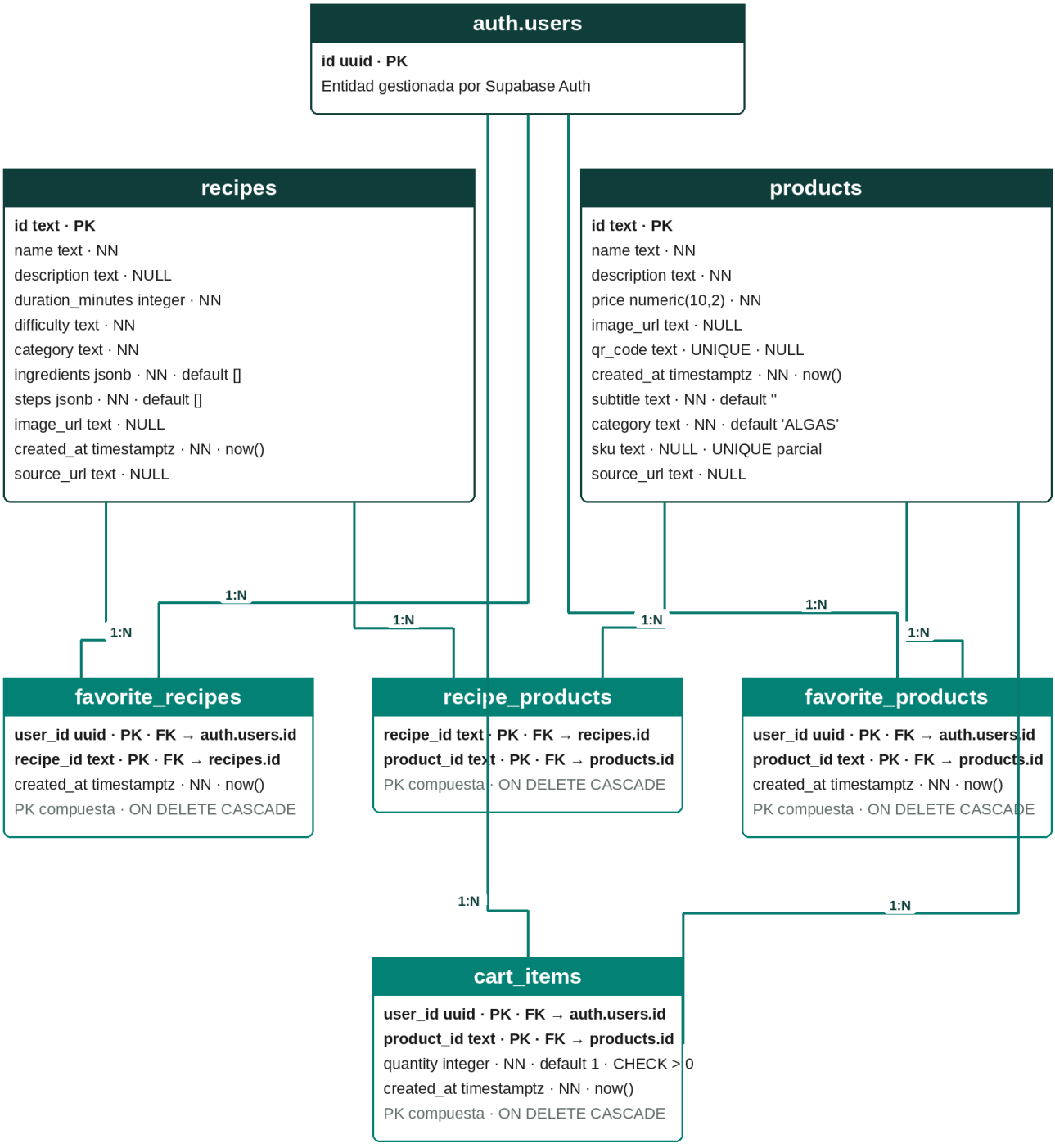
9.2. Modelo Entidad-Relación

El modelo de datos se diseñó a partir de dos grupos de información. Por un lado, products, recipes y recipe_products forman el catálogo público y describen contenido compartido por todos los usuarios. Por otro, favorite_products, favorite_recipes y cart_items almacenan estado privado vinculado a una identidad autenticada. Las cuentas no se duplican dentro del esquema public: se utiliza auth.users, tabla administrada por Supabase Auth, y sus identificadores UUID se referencian mediante claves foráneas. Este diseño evita replicar información de autenticación y mantiene separadas las responsabilidades del catálogo y del estado personal.

Las relaciones muchos-a-muchos se modelan mediante entidades asociativas. recipe_products conecta recetas y productos; favorite_products y favorite_recipes representan las selecciones de cada usuario; y cart_items relaciona usuarios y productos añadiendo un atributo propio, quantity. Esta normalización evita listas duplicadas dentro de una fila y permite aplicar integridad referencial y políticas RLS de forma precisa.

9.3. Diagrama E/R

La Figura 6 representa el esquema persistente utilizado por la versión final. Se incluyen las seis tablas del esquema public, la referencia a auth.users gestionada por Supabase Auth, los campos definidos en schema.sql, claves primarias y foráneas, restricciones principales y cardinalidades. En el modelo relacional físico las relaciones muchos-a-muchos no se dibujan como una conexión N:M directa: se resuelven mediante una tabla asociativa y, por ello, aparecen como dos relaciones 1:N. Así, recipes y products se relacionan mediante recipe_products; auth.users y recipes mediante favorite_recipes; y auth.users y products mediante favorite_products y cart_items.



Lectura de cardinalidades

El diagrama físico muestra relaciones 1:N hacia las tablas asociativas.
Las relaciones N:M conceptuales quedan resueltas de esta forma:
recipes ↔ products mediante recipe_products; auth.users ↔ recipes mediante favorite_recipes;
auth.users ↔ products mediante favorite_products y cart_items.

Figura 6. Diagrama Entidad-Relación completo del esquema persistente de Porto-Muiños App.

9.4. Descripción de tablas

Para facilitar la lectura y no concentrar todo el diccionario de datos en una única tabla extensa, cada entidad se documenta por separado. Las tablas siguientes reproducen todos los campos definidos en el esquema SQL del proyecto. PK indica clave primaria, FK clave foránea y NN (NOT NULL) obligatoriedad del valor. auth.users se trata como una entidad externa gestionada internamente por Supabase Auth; la aplicación únicamente referencia su identificador id desde las tablas privadas.

9.4.1. auth.users

Entidad de autenticación administrada por Supabase. No forma parte del esquema public creado por la aplicación; se representa porque sus identificadores se utilizan como claves foráneas en favoritos y carrito.

Campo	Tipo	Restricciones / valor por defecto	Finalidad
id	uuid	PK	Identificador único de la cuenta utilizado por las relaciones del esquema public.

Tabla 18. auth.users.

9.4.2. products

Tabla principal del catálogo de productos. Contiene la información comercial y de presentación necesaria para las fichas, la búsqueda, el QR y el acceso a la tienda oficial.

Campo	Tipo	Restricciones / valor por defecto	Finalidad
id	text	PK	Identificador estable del producto utilizado por la app y las relaciones.
name	text	NN	Nombre comercial mostrado al usuario.
description	text	NN	Descripción principal del producto.
price	numeric(10,2)	NN	Precio mostrado en catálogo, detalle y carrito.
image_url	text	NULL	Ruta del recurso gráfico o URL completa de imagen.
qr_code	text	UNIQUE · NULL	Valor asociado a un QR; evita asignar el mismo código a dos productos.
created_at	timestamptz	NN · default now()	Fecha de creación del registro.
subtitle	text	NN · default ''	Texto breve complementario utilizado en la interfaz.
category	text	NN · default 'ALGAS'	Categoría empleada para filtros y clasificación.
sku	text	NULL · índice UNIQUE parcial si no es NULL	Referencia comercial opcional sin duplicados cuando existe.
source_url	text	NULL	Enlace de referencia hacia el contenido o producto oficial.

Tabla 19. products.

9.4.3. recipes

Almacena el contenido estructurado de las recetas. Los ingredientes y pasos se conservan como arrays JSONB porque forman parte del propio contenido de la receta y se consumen como listas ordenadas desde la aplicación.

Campo	Tipo	Restricciones / valor por defecto	Finalidad
id	text	PK	Identificador estable de la receta.
name	text	NN	Título mostrado en listados y detalle.
description	text	NULL	Descripción o introducción opcional.
duration_minutes	integer	NN	Duración estimada en minutos.
difficulty	text	NN	Nivel de dificultad utilizado en la interfaz.
category	text	NN	Categoría empleada en filtros del recetario.
ingredients	jsonb	NN · default []::jsonb	Lista ordenada de ingredientes.
steps	jsonb	NN · default []::jsonb	Lista ordenada de pasos de preparación.
image_url	text	NULL	Ruta o URL de la imagen principal.
created_at	timestamptz	NN · default now()	Fecha de creación del registro.
source_url	text	NULL	Enlace de referencia al contenido original cuando procede.

Tabla 20. recipes.

9.4.4. recipe_products

Tabla asociativa que materializa la relación muchos-a-muchos entre recetas y productos. No necesita un identificador artificial porque la combinación receta-producto identifica de forma única cada asociación.

Campo	Tipo	Restricciones / valor por defecto	Finalidad
recipe_id	text	PK compuesta · FK → recipes.id · ON DELETE CASCADE	Receta participante en la relación.
product_id	text	PK compuesta · FK → products.id · ON DELETE CASCADE	Producto utilizado o asociado a la receta.

Tabla 21. recipe_products.

9.4.5. favorite_products

Registra los productos marcados como favoritos por cada usuario autenticado. La clave compuesta impide duplicar el mismo producto dentro de los favoritos de una misma cuenta.

Campo	Tipo	Restricciones / valor por defecto	Finalidad
user_id	uuid	PK compuesta · FK → auth.users.id · ON DELETE CASCADE	Cuenta propietaria del favorito.
product_id	text	PK compuesta · FK → products.id · ON DELETE CASCADE	Producto marcado como favorito.
created_at	timestamptz	NN · default now()	Momento en el que se creó la relación.

Tabla 22. favorite_products.

9.4.6. favorite_recipes

Registra las recetas favoritas de cada usuario. Sigue el mismo patrón relacional que favorite_products, manteniendo el estado personal separado del catálogo público.

Campo	Tipo	Restricciones / valor por defecto	Finalidad
user_id	uuid	PK compuesta · FK → auth.users.id · ON DELETE CASCADE	Cuenta propietaria del favorito.
recipe_id	text	PK compuesta · FK → recipes.id · ON DELETE CASCADE	Receta marcada como favorita.
created_at	timestamptz	NN · default now()	Momento en el que se creó la relación.

Tabla 23. favorite_recipes.

9.4.7. cart_items

Mantiene el carrito persistente de cada usuario. A diferencia de una asociación simple, incorpora quantity porque la relación usuario-producto necesita almacenar un dato propio: el número de unidades seleccionadas.

Campo	Tipo	Restricciones / valor por defecto	Finalidad
user_id	uuid	PK compuesta · FK → auth.users.id · ON DELETE CASCADE	Cuenta propietaria del carrito.
product_id	text	PK compuesta · FK → products.id · ON DELETE CASCADE	Producto añadido al carrito.
quantity	integer	NN · default 1 · CHECK (quantity > 0)	Número de unidades; la restricción impide cantidades nulas o negativas.
created_at	timestamptz	NN · default now()	Momento de creación inicial del elemento de carrito.

Tabla 24. cart_items.

9.5. Relaciones, cardinalidades e integridad referencial

products y recipes utilizan identificadores de tipo text porque el catálogo se referencia mediante códigos estables y legibles por la aplicación. Las tablas asociativas no incorporan un id artificial: utilizan claves primarias compuestas formadas por las dos claves que definen de manera natural cada relación. Por ejemplo, (recipe_id, product_id) impide asociar dos veces el mismo producto a una receta, mientras que (user_id, product_id) evita duplicar un favorito o una línea de carrito para el mismo usuario y producto.

La integridad se refuerza mediante restricciones adicionales. qr_code es UNIQUE para impedir que dos productos resuelvan el mismo código; sku dispone de un índice único parcial que solo se aplica cuando el valor no es NULL; y cart_items exige quantity > 0 mediante CHECK. Las listas variables de ingredients y steps se almacenan como JSONB: PostgreSQL mantiene una representación JSON estructurada y consultable sin obligar a crear una tabla adicional para un contenido cuyo orden y estructura pertenecen a cada receta.

Todas las claves foráneas de las tablas asociativas emplean ON DELETE CASCADE. Si se elimina un usuario, producto o receta, PostgreSQL elimina automáticamente las filas dependientes correspondientes. De este modo se evitan registros huérfanos y la coherencia no depende de que el cliente Android ejecute manualmente una cadena de borrados.

Relación conceptual	Cardinalidad	Tabla / claves que la implementan	Comportamiento
Recetas ↔ Productos	N:M	recipe_products (recipe_id, product_id)	Una receta puede usar varios productos y un producto aparecer en varias recetas.
Usuarios ↔ Productos favoritos	N:M	favorite_products (user_id, product_id)	Cada usuario conserva su conjunto privado de productos favoritos.
Usuarios ↔ Recetas favoritas	N:M	favorite_recipes (user_id, recipe_id)	Cada usuario conserva su conjunto privado de recetas favoritas.
Usuarios ↔ Productos del carrito	N:M con atributo	cart_items (user_id, product_id) + quantity	La relación almacena además la cantidad elegida para cada producto.

Tabla 25. Relaciones, cardinalidades y tablas asociativas del modelo.

9.6. Row Level Security

Row Level Security (RLS) constituye la principal barrera de autorización a nivel de datos. El catálogo —products, recipes y recipe_products— puede consultarse públicamente, pero favoritos y carrito están restringidos al usuario autenticado. Las políticas comparan auth.uid() con user_id para SELECT, INSERT, UPDATE o DELETE según corresponda. [14]

Este enfoque es importante porque la seguridad no depende únicamente de que la interfaz oculte datos ajenos. Aunque un cliente intentase construir manualmente una petición sobre una fila de otro usuario, PostgreSQL evaluaría la política RLS y denegaría la operación. La protección permanece por tanto en el backend. [14]

9.7. Supabase Auth

La autenticación soporta registro mediante correo y contraseña, confirmación de correo, inicio de sesión, persistencia de sesión, cierre de sesión, recuperación de contraseña y eliminación de cuenta. El nombre completo se guarda como metadato de usuario y se utiliza para personalizar Home y Perfil. La inicialización de Auth se espera durante Splash para decidir si existe una sesión válida antes de navegar. [12]

9.8. Google OAuth

Google OAuth se integra a través del proveedor Google de Supabase Auth. El botón “Continuar con Google” cubre tanto alta como acceso: si la identidad todavía no existe, Supabase crea el usuario; si ya existe, reutiliza la cuenta. El retorno de OAuth utiliza el esquema de deep link portomuinos://auth y la aplicación procesa el resultado antes de navegar a Home. [13]

La contraseña de Google nunca es recibida por la aplicación. Una cuenta creada exclusivamente mediante OAuth se autentica con Google salvo que posteriormente se implemente un flujo específico para establecer una contraseña, considerado como posible mejora futura.

9.9. Storage

Las imágenes del catálogo se almacenan en un bucket público denominado catalog-media. El mapper acepta tanto rutas internas del bucket como URLs completas; cuando recibe una ruta construye la URL pública a partir de SUPABASE_URL. El bucket está limitado a imágenes JPEG, PNG y WebP y dispone de un límite de archivo de 5 MiB, suficiente para el conjunto actual de recursos de catálogo. [15]

9.10. Edge Function delete-account

Eliminar un usuario de Auth requiere privilegios administrativos que no deben incorporarse dentro de una aplicación cliente. Por este motivo se implementó la Edge Function delete-account. La función exige una sesión de usuario válida, obtiene el identificador de la identidad autenticada y ejecuta la eliminación mediante el cliente administrativo de Supabase en el entorno servidor. [16]

La aplicación Android invoca únicamente la función autenticada y, tras una respuesta correcta, limpia la sesión local. De esta forma ninguna service_role key ni credencial administrativa se distribuye dentro del APK.

9.11. Gestión de credenciales

Las credenciales de conexión de desarrollo se mantienen en local.properties y se exponen al código mediante campos BuildConfig. Este archivo no debe distribuirse ni incluirse en el repositorio. Para facilitar la reproducción del proyecto se conserva local.properties.example con los nombres de las variables requeridas, pero sin valores reales.

La clave incluida en la aplicación corresponde al acceso público previsto para clientes y su capacidad queda limitada por RLS. Las operaciones administrativas, como eliminar una cuenta, se ejecutan en el backend. Esta separación evita que secretos de servidor formen parte del código Android.

10. Tecnologías, herramientas, APIs y librerías

10.1. Kotlin

Kotlin es el lenguaje principal del proyecto Android. Se eligió por ser la opción moderna de referencia para desarrollo Android, por su interoperabilidad con el ecosistema Java y por características como null safety, coroutines, extensiones y data classes. Estas capacidades resultan especialmente útiles para modelar estados de UI y operaciones asíncronas contra Supabase. [9]

10.2. Jetpack Compose

Jetpack Compose se utiliza para construir toda la interfaz mediante funciones @Composable. El enfoque declarativo permite que la UI dependa del estado expuesto por los ViewModels y reduce la necesidad de mantener layouts XML y sincronizaciones manuales. Material 3 se utiliza como base de componentes, aunque la identidad visual se personaliza con colores, tipografía, formas y componentes propios. [7]

10.3. Android Studio

Android Studio es el entorno principal de desarrollo, compilación y depuración. Se utilizaron el emulador Pixel 9 Pro XL, Logcat, inspecciones estáticas, Asset Studio para el icono launcher y las herramientas de Build/Gradle. La fase final incluyó además Inspect Code sobre el proyecto completo para localizar código redundante, recursos no utilizados y recomendaciones de Android Lint.

10.4. Gradle y JDK 21

Gradle gestiona la compilación y dependencias. Durante la validación final se utilizó el JDK 21 incluido con Android Studio para ejecutar Gradle 8.13 y los tests unitarios. El módulo Android mantiene compatibilidad de compilación JVM objetivo 11, compileSdk y targetSdk 36 y minSdk 26. La separación entre JDK de ejecución de Gradle y bytecode objetivo evita confundir la herramienta de construcción con la versión mínima de Android. [10]

10.5. Supabase

Supabase aporta backend relacional, autenticación, Storage, APIs automáticas y funciones serverless en un único servicio. La aplicación utiliza el SDK Kotlin 2.6.0 con los módulos PostgREST, Auth y Functions. Su elección permitió disponer de persistencia multiusuario y políticas de seguridad sin desarrollar y desplegar una API REST propia para el alcance académico. [11]

10.6. PostgreSQL

PostgreSQL es la base de datos relacional que sustenta Supabase. Su modelo permite representar de forma directa productos, recetas, relaciones muchos-a-muchos, favoritos y carrito. La utilización de claves foráneas, restricciones, índices y Row Level Security traslada integridad y autorización al servidor en lugar de depender exclusivamente del cliente móvil. [17]

10.7. CameraX

CameraX proporciona el acceso a la cámara dentro de QrScannerScreen y simplifica la relación con el ciclo de vida de Android. Se integran los módulos core, camera2, lifecycle, view y camera-mlkit-vision, todos en la versión 1.6.1 del proyecto auditado. La cámara se utiliza exclusivamente durante el flujo de lectura QR. [18]

10.8. ML Kit

Google ML Kit Barcode Scanning 17.3.0 procesa los frames recibidos desde CameraX y detecta códigos QR. El valor leído se entrega a QrProductResolver, que normaliza varios formatos admitidos —identificador interno, URI propia o URL— y determina qué producto debe abrirse dentro de la aplicación. [19]

10.9. Coil

Coil Compose 2.7.0 se utiliza para cargar imágenes remotas del catálogo dentro de Compose. CatalogImage encapsula esta responsabilidad y permite mostrar tanto recursos locales como URLs provenientes de Supabase Storage, manteniendo el tratamiento de imágenes fuera de las pantallas que consumen el componente. [20]

10.10. Kotlin Serialization y Ktor

Kotlin Serialization permite transformar las respuestas JSON del backend en DTOs tipados. Ktor Client Android 2.3.12 actúa como motor HTTP utilizado por las librerías de Supabase. Esta combinación evita manipular JSON manualmente y mantiene los contratos de datos representados mediante clases Kotlin. [21][22]

10.11. Figma

Figma se empleó durante el diseño y prototipado de la interfaz. Permitió definir composiciones, componentes, navegación inferior, jerarquía visual y mockups antes de su implementación. El resultado final no reproduce de forma mecánica cada prototipo: la validación en Android introdujo ajustes necesarios para estados reales, navegación, persistencia y adaptación a dispositivos.

10.12. Justificación de las tecnologías seleccionadas

La selección tecnológica busca un equilibrio entre calidad, mantenibilidad y alcance. Kotlin y Compose permiten una aplicación Android nativa con una UI declarativa; MVVM y Repository ordenan responsabilidades; Supabase proporciona un backend relacional y seguro sin mantener un servidor propio; CameraX y ML Kit resuelven la integración QR con APIs específicas de Android; y Coil simplifica la carga de contenido gráfico remoto.

Tecnología / herramienta	Uso en Porto-Muiños App	Motivo principal de elección
Kotlin	Lenguaje de toda la aplicación Android.	Integración nativa con Android, seguridad de tipos y coroutines.
Jetpack Compose + Material 3	Interfaz y componentes.	UI declarativa, reutilización y adaptación al estado.
Navigation Compose	Grafo de pantallas y argumentos.	Integración directa con Compose.
Supabase 2.6.0	Auth, PostgREST, Functions y conexión backend.	Backend completo basado en PostgreSQL con RLS.
PostgreSQL	Persistencia relacional.	Integridad, relaciones, restricciones y seguridad por filas.
CameraX 1.6.1 + ML Kit 17.3.0	Escaneo QR.	Integración de cámara y reconocimiento de códigos con ciclo de vida Android.
Coil Compose 2.7.0	Imágenes remotas.	Carga asíncrona sencilla en Compose.
Kotlin Serialization + Ktor 2.3.12	Serialización y transporte HTTP.	DTOs tipados e integración con SDK Supabase.
Figma	Diseño y prototipado.	Definición visual previa y revisión de UX.
ChatGPT	Apoyo en contraste de decisiones, depuración, revisión y documentación.	Herramienta de asistencia; no forma parte de la ejecución de la app.

Tabla 26. Tecnologías y herramientas principales y su función en el proyecto.

11. Desarrollo e implementación

11.1. Autenticación

El sistema de autenticación se implementa sobre Supabase Auth y está encapsulado por AuthRepository y AuthViewModel. Esta separación evita que las pantallas Compose dependan directamente del SDK remoto y concentra en una única capa las operaciones de registro, inicio y cierre de sesión, consulta del usuario actual y eliminación de cuenta.

El registro tradicional utiliza correo electrónico y contraseña. Durante el alta se almacena además el nombre completo en los metadatos del usuario mediante la clave full_name. Supabase determina si la sesión puede iniciarse inmediatamente o si es necesaria la confirmación previa del correo; la interfaz refleja ambos casos con mensajes comprensibles y estados de carga.

El inicio de sesión valida las credenciales mediante Supabase y, cuando finaliza correctamente, actualiza el nombre mostrado y restaura los datos asociados al usuario. La sesión se conserva mediante el mecanismo propio de Supabase Auth, por lo que la aplicación puede reconocer una sesión válida al iniciarse de nuevo. El cierre de sesión

elimina únicamente el estado de sesión y los estados temporales de interfaz, mientras que la eliminación de cuenta se delega en una Edge Function con privilegios de servidor.

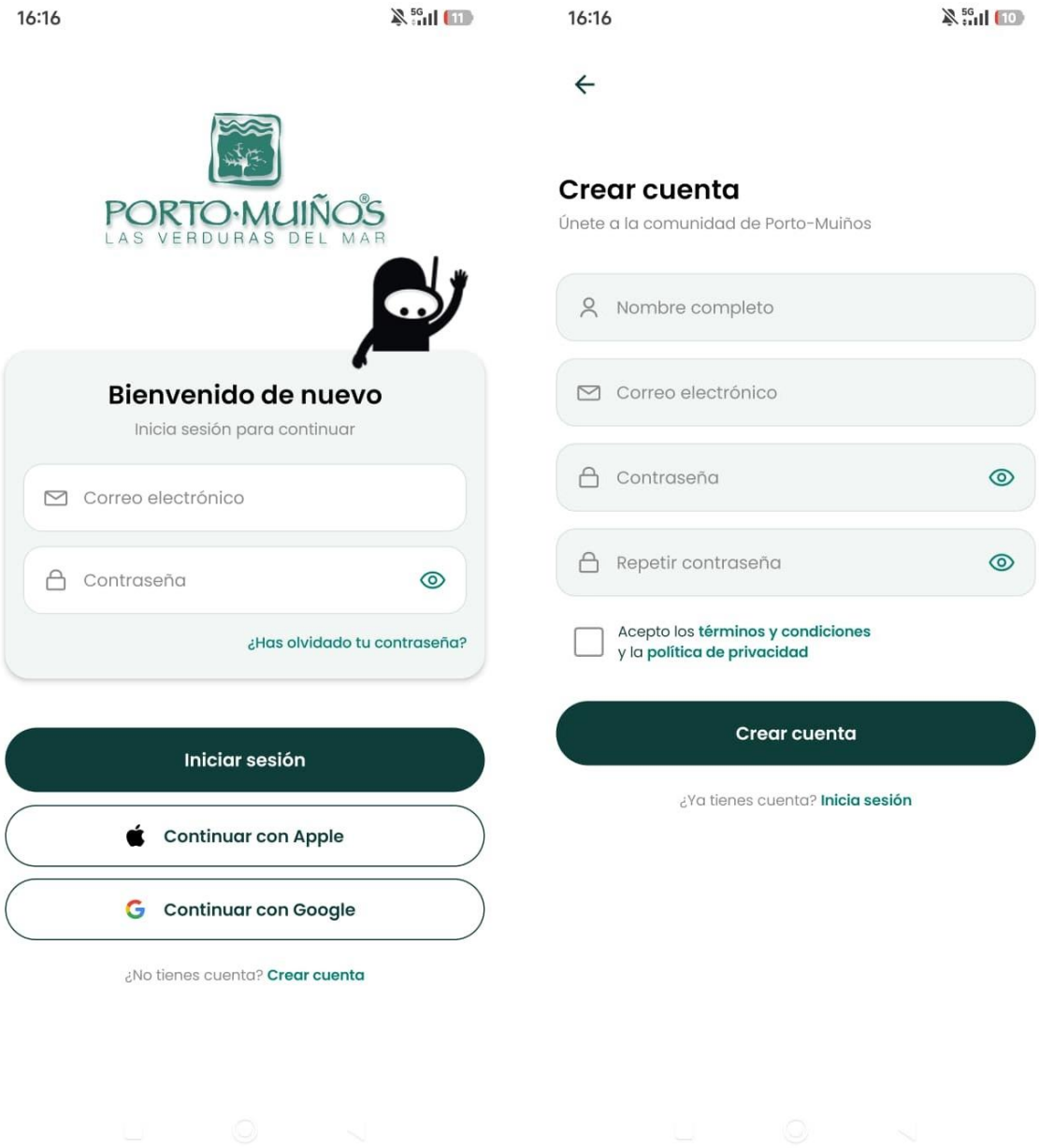


Figura 7. Pantallas de autenticación por correo: inicio de sesión y creación de cuenta.

11.2. Recuperación de contraseña y deep links

La recuperación de contraseña utiliza el flujo de correo electrónico de Supabase. Desde la pantalla correspondiente se solicita el envío de un enlace de recuperación y se evita revelar si una dirección concreta existe o no, mostrando un mensaje neutro al usuario.

Para devolver el control a la aplicación se configuró el deep link portomuinos://auth. MainActivity entrega el enlace al cliente de Supabase y AuthDeepLinkManager distingue entre una recuperación de contraseña y un retorno de autenticación externa. La navegación consume posteriormente ese estado y dirige al usuario a la pantalla adecuada.

En una recuperación válida, el usuario establece una nueva contraseña mediante AuthRepository.updatePassword(). Tras completar la operación se cierra la sesión temporal de recuperación y se vuelve al acceso normal, evitando mantener un contexto sensible abierto innecesariamente.

11.3. Google OAuth

Google OAuth se integra como método alternativo de autenticación. AuthRepository inicia el proveedor Google y utiliza el mismo deep link de retorno de la aplicación. El proceso real de identificación se realiza en el entorno seguro de Google y Supabase; la aplicación no recibe ni almacena la contraseña de la cuenta de Google.

El mismo botón permite continuar tanto a usuarios ya existentes como a usuarios que todavía no tienen una identidad Google registrada en el proyecto. En este último caso, Supabase crea la cuenta automáticamente tras una autenticación válida. Esta característica explica por qué el texto de la interfaz se formula como «Continuar con Google» y no únicamente como «Iniciar sesión».

La opción de Apple se conserva visualmente como ampliación futura y comunica de forma explícita que todavía no está disponible. De este modo se mantiene la estructura prevista de la interfaz sin simular una funcionalidad que no forma parte de la versión entregada.

11.4. Catálogo de productos

El catálogo se obtiene de Supabase mediante CatalogRemoteDataSource. Los registros remotos se reciben como DTO y CatalogMapper los transforma en modelos de interfaz, evitando que las pantallas dependan del formato exacto de la base de datos.

CatalogRepository reúne productos, recetas y relaciones receta-producto en un único objeto CatalogData. CatalogViewModel realiza la carga remota y mantiene el resultado

como estado observable para que diferentes pantallas puedan reutilizar la misma fuente de información.

Cada producto dispone de los datos necesarios para su presentación y navegación: identificador, nombre, subtítulo, descripción, precio, categoría, imagen, código QR, SKU y enlace de origen cuando corresponde. Las imágenes se cargan de forma remota y las fichas reutilizan la información del catálogo sin duplicarla en la interfaz.

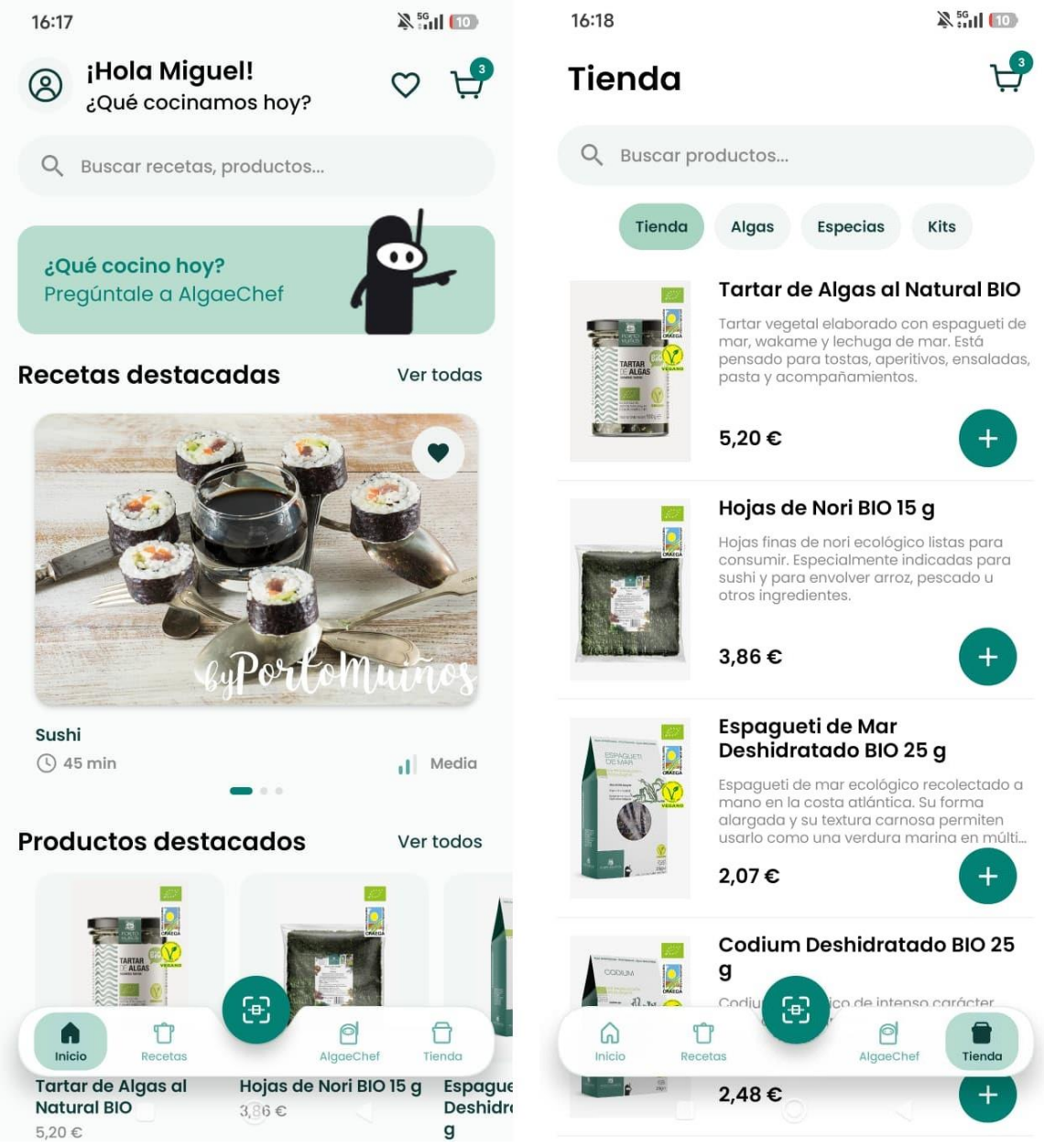


Figura 8. Pantallas de Inicio y Tienda utilizadas para la exploración del catálogo.

11.5. Recetario

El recetario utiliza las recetas almacenadas en Supabase y las presenta mediante un listado filtrable. Cada receta incluye información de contexto, duración, dificultad, categoría, ingredientes, pasos, imagen y enlace de origen cuando está disponible.

La pantalla de detalle separa ingredientes y preparación paso a paso, mantiene el acceso a favoritos y muestra los productos Porto-Muiños relacionados. El objetivo es que una receta no sea un contenido aislado, sino un punto de entrada hacia los productos necesarios para reproducirla.

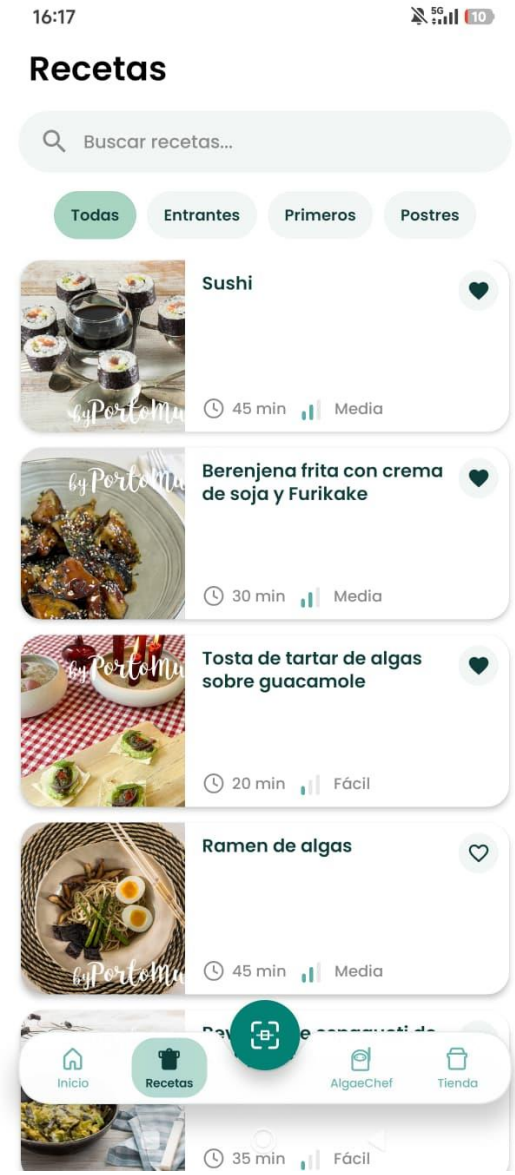


Figura 9. Pantalla de Recetas con buscador, filtros, metadatos y favoritos.

11.6. Buscadores y filtros

La búsqueda se implementa en una utilidad independiente, CatalogSearch, para poder reutilizar la misma lógica en productos y recetas. Antes de comparar textos se aplica una normalización que elimina diacríticos, convierte a minúsculas y recorta espacios, de modo que consultas como «atlantica» puedan encontrar «atlántica».

En productos se consulta el nombre, subtítulo, descripción y categoría; en recetas se incluyen título, descripción, categoría e ingredientes. Los filtros por categoría se aplican sobre los resultados mostrados y permiten reducir el catálogo sin modificar los datos almacenados.

11.7. Relaciones producto-receta

La relación entre productos y recetas se materializa mediante la tabla recipe_products. Esta tabla asociativa permite que una receta utilice varios productos y que un mismo producto aparezca en varias recetas sin duplicar información.

En la aplicación, estas relaciones se transforman en RecipeProductRelationUi y se utilizan para construir navegación bidireccional: desde una receta pueden consultarse los productos asociados y desde la ficha de un producto se muestran recetas relacionadas. Esta conexión constituye uno de los elementos centrales de la propuesta funcional.



Figura 10. Detalle de receta con ingredientes y producto Porto-Muiños relacionado.

11.8. Favoritos

Los favoritos se almacenan de forma persistente en favorite_products y favorite_recipes, siempre asociados al identificador del usuario autenticado. FavoriteRepository encapsula las lecturas, inserciones y eliminaciones, mientras que FavoritesViewModel mantiene en memoria los conjuntos de identificadores visibles para la interfaz.

La actualización es optimista: al pulsar el icono de favorito la interfaz cambia de forma inmediata y la escritura remota se ejecuta a continuación. Las operaciones repetidas sobre un mismo elemento se encadenan mediante Jobs para conservar el orden. Si Supabase devuelve un error y el estado mostrado todavía corresponde a la operación fallida, el ViewModel restaura el valor anterior y comunica la incidencia.

La pantalla de favoritos combina posteriormente los identificadores persistidos con el catálogo ya cargado, evitando guardar copias completas de productos o recetas en las tablas de relación.

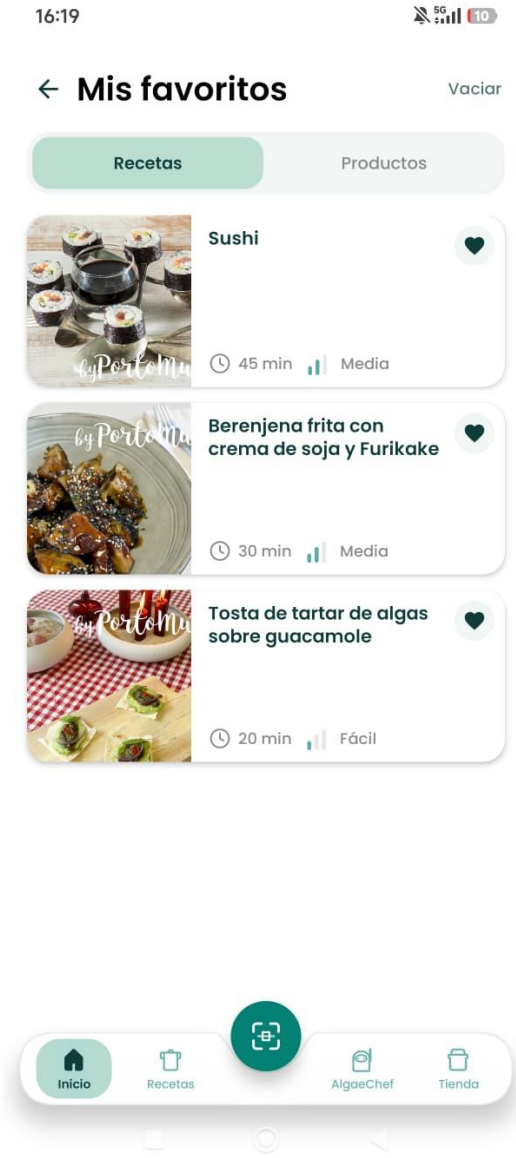


Figura 11. Pantalla de favoritos persistentes del usuario.

11.9. Carrito

El carrito persistente utiliza cart_items, donde cada fila relaciona un usuario con un producto y almacena además la cantidad. CartRepository permite leer el carrito, establecer una cantidad mediante upsert, eliminar un producto o vaciar todos los elementos del usuario.

CartViewModel representa el estado como un mapa productId-cantidad y aplica también actualización optimista. Para evitar que pulsaciones rápidas sobre los controles «+» y «-» generen escrituras remotas fuera de orden, las operaciones se serializan: cada sincronización espera a que termine la anterior antes de ejecutarse.

Cuando una operación falla, el ViewModel intenta recuperar el estado remoto más reciente; si tampoco es posible, restaura la cantidad previa conservada localmente. Esta estrategia mantiene una respuesta inmediata de la interfaz sin renunciar a la consistencia con Supabase.

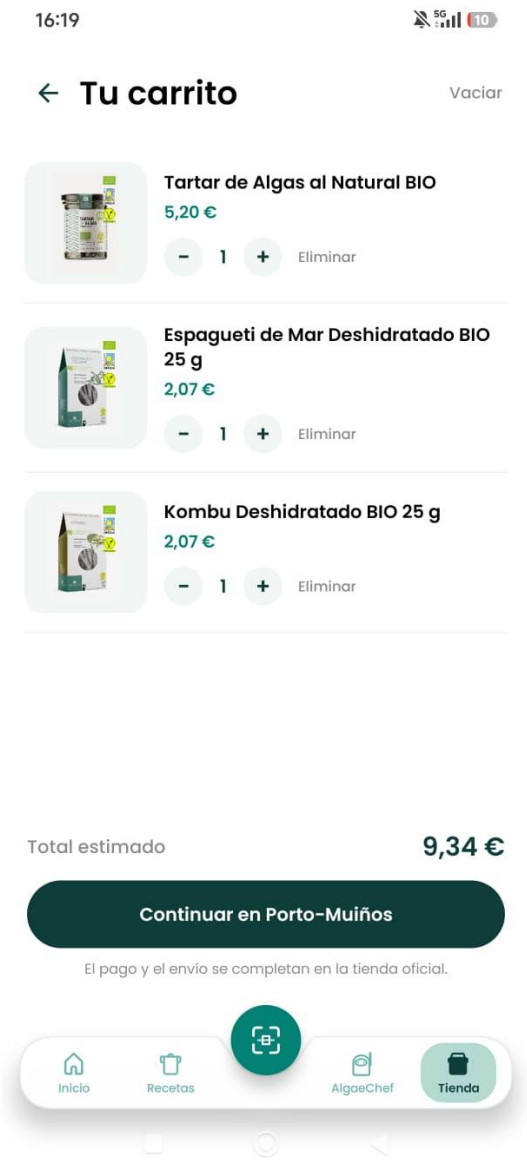


Figura 12. Carrito persistente con cantidades, eliminación y total estimado.

11.10. Integración con la tienda oficial

La aplicación no implementa un sistema propio de pagos. Los productos pueden contener source_url para abrir su referencia correspondiente en la web oficial, y desde el carrito se ofrece una acción para continuar en la tienda de Porto-Muiños.

Esta decisión evita duplicar funciones de comercio electrónico ya existentes y reduce el alcance de seguridad, pagos, logística y cumplimiento asociado a un checkout propio. El carrito de la aplicación sirve como herramienta de selección y preparación de compra; el pago y el envío se completan en el canal oficial.

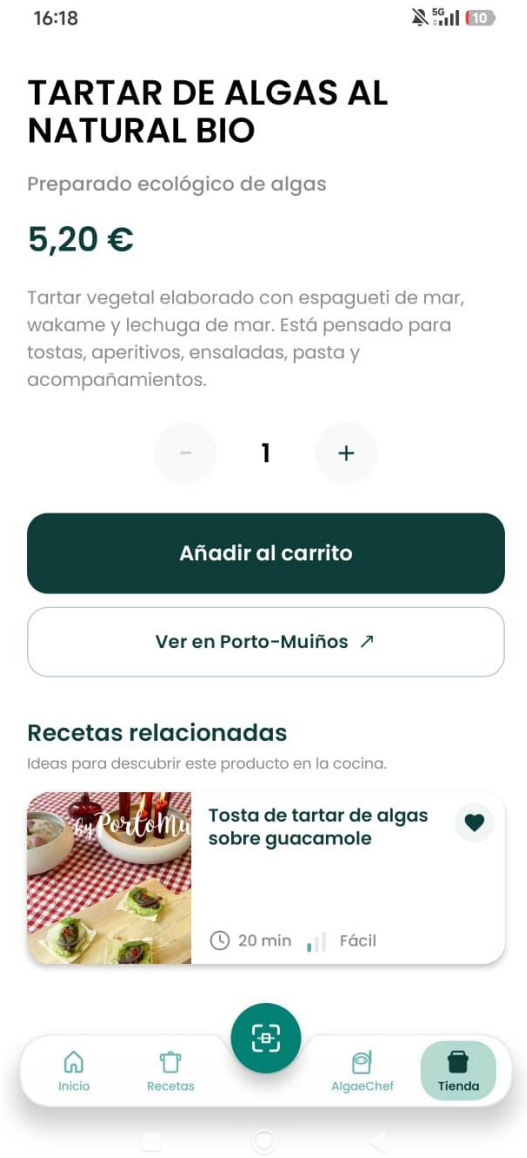


Figura 13. Detalle de producto con acceso a Porto-Muiños y receta relacionada.

11.11. Escáner QR

El escáner utiliza CameraX para gestionar la cámara y ML Kit Barcode Scanning para detectar códigos QR en tiempo real. Una vez obtenido el contenido, QrProductResolver lo normaliza y busca coincidencias contra los productos cargados desde Supabase.

El resolver admite distintos formatos: identificadores internos, rutas, deep links, URLs completas y parámetros como productId, product_id, id o qr_code. La comparación se realiza tanto contra el id del producto como contra el valor almacenado en qr_code, lo que permite compatibilizar códigos de prueba con URLs oficiales existentes.

Cuando se identifica un producto, la aplicación navega a su ficha interna en lugar de abandonar inmediatamente el entorno móvil. De este modo el QR actúa como puente entre el envase físico y las recetas, información y opciones de compra disponibles en la app.



Figura 14. Escaneo de un QR real y detección del producto desde un dispositivo físico.

11.12. AlgaeChef

AlgaeChef se implementa como demostrador conversacional integrado en el mismo patrón de estado que el resto de la aplicación. El usuario puede escribir una petición libre o elegir opciones rápidas como «Tengo pescado», «Algo rápido», «Vegetariano», «Quiero algo ligero» o «Para sorprender».

La versión actual no utiliza un modelo generativo externo. generateMockAlgaeChefResult aplica reglas deterministas sobre el texto introducido y selecciona una receta entre las recetas reales cargadas desde Supabase. Por ejemplo, una petición rápida prioriza la receta de menor duración, mientras que las consultas de pescado, vegetarianas o especiales buscan palabras relacionadas en título, descripción e ingredientes.

Aunque la respuesta conversacional sea simulada, el identificador de receta devuelto pertenece al catálogo real. Por ello, la recomendación puede abrir una receta válida y continuar el flujo normal de favoritos, productos relacionados y navegación. AlgaeChefViewModel mantiene la conversación, el estado «pensando» y la secuencia de mensajes, dejando preparada la interfaz para sustituir en el futuro el generador determinista por un proveedor de IA.



Figura 15. Ejemplo de recomendación generada por el demostrador AlgaeChef.

11.13. Perfil

La pantalla de perfil reúne la identidad básica del usuario, la imagen de perfil seleccionada, el número de favoritos y las acciones relacionadas con la cuenta. El nombre procede de los metadatos de Supabase Auth, con un valor alternativo derivado del correo cuando no existe full_name.

La imagen elegida por el usuario se gestiona como preferencia local, ya que no es un dato imprescindible para el funcionamiento remoto de la aplicación. La sección incluye acceso a favoritos, cierre de sesión y eliminación de cuenta.

Otros apartados visibles, como preferencias alimentarias, alergias e intolerancias o ajustes avanzados, muestran de forma deliberada la estructura prevista para futuras ampliaciones. Se presentan como parte del diseño de producto, pero no se documentan como funcionalidades implementadas.

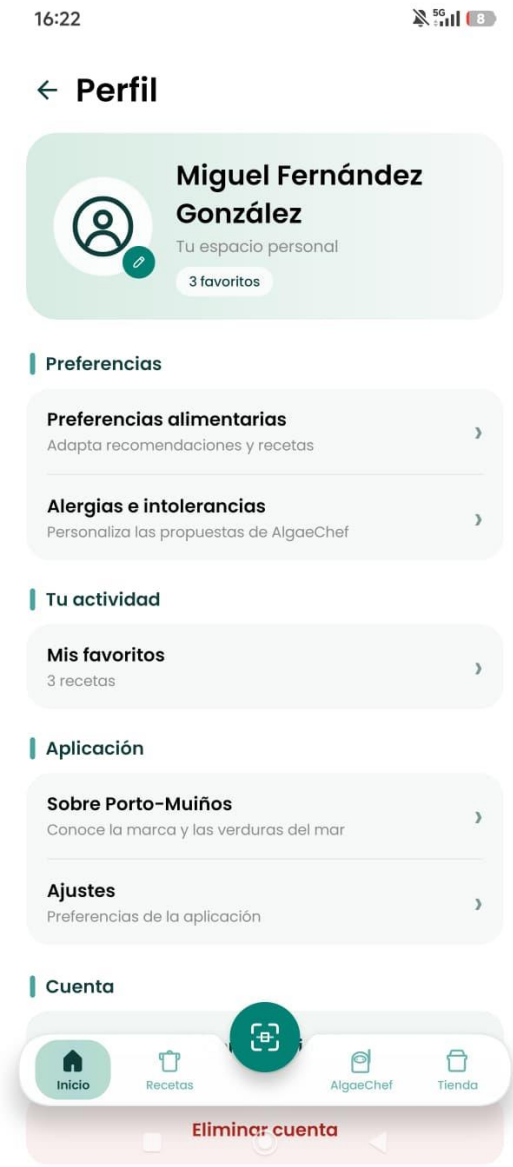


Figura 16. Pantalla de perfil con información de usuario, favoritos y opciones de cuenta.

11.14. Persistencia y sincronización de datos

La aplicación combina persistencia remota y local según la naturaleza de cada dato. Supabase Auth mantiene la sesión; PostgreSQL almacena catálogo, relaciones, favoritos y carrito; y las preferencias sencillas del dispositivo se reservan para elementos como onboarding e imagen de perfil.

Al restaurar una sesión válida se vuelven a cargar los datos asociados al usuario. Al finalizarla se limpian los estados de presentación que no deben permanecer activos entre sesiones. Este reparto evita convertir SharedPreferences en una base de datos alternativa y mantiene en Supabase la información que debe ser consistente entre sesiones y protegida por RLS.

Tipo de información	Persistencia	Responsabilidad
Sesión autenticada	Supabase Auth	Restaurar y validar la sesión del usuario.
Productos, recetas y relaciones	PostgreSQL / Supabase	Fuente remota común para el catálogo.
Favoritos	PostgreSQL / Supabase	Persistencia por usuario con RLS.
Carrito y cantidades	PostgreSQL / Supabase	Persistencia por usuario y sincronización ordenada.
Onboarding e imagen de perfil	Preferencias locales	Estado sencillo ligado al dispositivo.
Conversación AlgaeChef	Estado temporal del ViewModel	Se reinicia al comenzar una nueva sesión conversacional.

Tabla 27. Estrategia de persistencia utilizada según el tipo de información.

12. Calidad del código y buenas prácticas

12.1. Separación de responsabilidades

La calidad estructural del proyecto se apoya en la separación entre presentación, estado y acceso a datos descrita en el capítulo 8. Las pantallas Compose se centran en representar información y emitir acciones; los ViewModels gestionan estado y lógica de presentación; y los repositories encapsulan las operaciones de Supabase.

Esta organización evita llamadas directas a la base de datos desde las pantallas y reduce las dependencias cruzadas. La navegación se mantiene fuera de los ViewModels y se coordina desde PortoNavHost, lo que permite que las clases de estado no necesiten conocer NavController ni destinos concretos.

12.2. Nomenclatura y estilo

La estructura de paquetes sigue responsabilidades reconocibles: data/local, data/remote, data/repository, ui/components, ui/model, ui/navigation, ui/screens, ui/search, ui/qr y ui/viewmodel. Los nombres de clases reflejan su papel mediante sufijos consistentes como Screen, ViewModel, Repository, Dto o Ui.

Durante la auditoría final se eliminaron imports comodín y imports confirmados como no utilizados, tabs, espacios finales y referencias históricas de depuración. También se revisó la ortografía de los nombres de pruebas y se mantuvo un formato homogéneo en los archivos Kotlin.

12.3. Comentarios y documentación interna

Todos los archivos Kotlin de producción incluyen una cabecera breve que indica su responsabilidad. En clases con mayor lógica se añaden comentarios de sección para separar carga, acciones públicas, sincronización o utilidades internas.

Se evita comentar instrucciones evidentes línea por línea. Los comentarios se reservan para explicar responsabilidades, decisiones no triviales o motivos de diseño, de modo que sirvan como apoyo de mantenimiento sin duplicar el propio código.

12.4. Refactorización a MVVM

Durante el desarrollo inicial, PortoMuinosApp concentraba una parte importante del estado y de las operaciones de sesión, catálogo, favoritos y carrito. Una vez estabilizada la funcionalidad se realizó una refactorización progresiva a cinco ViewModels especializados: AuthViewModel, CatalogViewModel, FavoritesViewModel, CartViewModel y AlgaeChefViewModel.

La refactorización se realizó preservando el comportamiento visible. Cada bloque se trasladó a su nueva responsabilidad y se verificó después mediante compilación, pruebas unitarias y pruebas funcionales. El resultado reduce el tamaño conceptual de la raíz de Compose y facilita modificar una funcionalidad sin alterar las restantes.

12.5. Eliminación de código y recursos obsoletos

La fase de cierre incluyó la revisión de imports, declaraciones, archivos temporales y restos de versiones previas. Se eliminaron referencias de depuración y elementos cuya ausencia no modificaba el comportamiento funcional.

No se forzó, sin embargo, la eliminación de todos los avisos de Android Studio. Algunas advertencias corresponden a versiones más recientes de librerías, recomendaciones KTX, recursos de plantilla o sugerencias de internacionalización que no justificaban introducir cambios de última hora en una versión estable.

12.6. Auditoría con Android Studio

La revisión final combinó compilación, pruebas unitarias y la herramienta Inspect Code de Android Studio. La inspección permitió identificar advertencias de corrección, rendimiento, productividad, usabilidad, internacionalización y código redundante.

Se priorizaron los problemas con efecto real sobre compilación, comportamiento o mantenibilidad. Entre los avisos no bloqueantes quedaron recomendaciones de actualización de dependencias, optimización de un vector complejo, recursos gráficos situados en drawable sin variantes de densidad, uso opcional de extensiones KTX y comprobaciones ortográficas sobre textos españoles.

Durante la ejecución de pruebas desde PowerShell se detectó además que Gradle estaba utilizando inicialmente JDK 25. El proyecto funcionaba desde Android Studio con su JBR 21, por lo que se configuró JAVA_HOME temporalmente hacia ese JDK y la tarea testDebugUnitTest finalizó correctamente. Esta incidencia confirmó la conveniencia de documentar JDK 21 como entorno de ejecución de Gradle.

12.7. Seguridad

Las credenciales de conexión a Supabase no se escriben directamente en el código fuente. Se leen desde local.properties y se exponen al módulo mediante BuildConfig; el repositorio incluye un local.properties.example sin secretos para documentar las variables necesarias.

La protección de los datos por usuario se completa mediante Row Level Security en las tablas privadas. La aplicación utiliza únicamente la clave cliente prevista para el entorno móvil; las operaciones administrativas, como eliminar definitivamente una cuenta, se ejecutan en una Edge Function y no exponen credenciales de servicio en el APK.

Las contraseñas y la autenticación social son gestionadas por Supabase Auth y los proveedores correspondientes. La app no almacena contraseñas en preferencias locales ni implementa un sistema criptográfico propio.

Medida de calidad	Aplicación en el proyecto
Arquitectura	MVVM + Repository y navegación separada de los ViewModels.
Nomenclatura	Sufijos y paquetes coherentes con cada responsabilidad.
Comentarios	Cabecera de responsabilidad y comentarios solo en bloques no evidentes.
Limpieza	Imports comodín/obsoletos, restos de depuración y TODO/FIXME revisados.
Seguridad	RLS, Auth, secretos fuera del código y Edge Function administrativa.
Validación	Assemble, 13 tests unitarios, pruebas manuales e Inspect Code.

Tabla 28. Principales medidas de calidad aplicadas en la versión final.

13. Pruebas y validación

13.1. Estrategia de pruebas

La validación se plantea en varias capas complementarias. Las funciones deterministas con lógica fácilmente aislable se cubren mediante pruebas unitarias JVM. Los flujos que dependen de Supabase, OAuth, cámara, navegación o almacenamiento se verifican mediante pruebas funcionales sobre la aplicación completa.

Además, se realizan pruebas tanto en emulador como en un dispositivo Android físico y una inspección estática final con Android Studio. Esta combinación resulta más representativa para el alcance del proyecto que limitar la validación únicamente a pruebas automatizadas.

13.2. Pruebas unitarias

La versión final contiene 13 pruebas unitarias distribuidas en tres clases. CatalogSearchTest valida la normalización y búsqueda; AuthErrorMapperTest comprueba la traducción de errores técnicos de autenticación a mensajes de usuario; y ProductFormattingTest valida el formato monetario empleado por los precios.

Las pruebas se ejecutaron mediante la tarea Gradle testDebugUnitTest con JDK 21. La ejecución final devolvió BUILD SUCCESSFUL, con 26 tareas accionables (4 ejecutadas y 22 ya actualizadas).

Suite	Caso validado	Resultado
CatalogSearchTest	Consulta vacía coincide con cualquier producto.	Superado
CatalogSearchTest	Búsqueda de productos ignora mayúsculas y tildes.	Superado
CatalogSearchTest	Coincidencia en descripción del producto.	Superado
CatalogSearchTest	Coincidencia por categoría del producto.	Superado
CatalogSearchTest	Búsqueda de recetas por ingrediente ignorando tildes.	Superado
CatalogSearchTest	Consulta no relacionada no produce coincidencia.	Superado
AuthErrorMapperTest	Credenciales incorrectas generan mensaje comprensible.	Superado
AuthErrorMapperTest	Correo no confirmado genera mensaje específico.	Superado
AuthErrorMapperTest	Límite de envío de correos se traduce correctamente.	Superado
AuthErrorMapperTest	Error de red genera mensaje de conexión.	Superado
AuthErrorMapperTest	Error desconocido utiliza mensaje alternativo.	Superado
ProductFormattingTest	Precio utiliza coma decimal y símbolo de euro.	Superado
ProductFormattingTest	Precio mantiene dos decimales.	Superado

Tabla 29. Pruebas unitarias ejecutadas en la versión final.

13.3. Pruebas funcionales

Las pruebas funcionales se realizaron sobre una compilación completa conectada al proyecto real de Supabase. Se comprobó la navegación principal, el tratamiento de estados de carga y error, la recuperación tras cerrar y abrir la aplicación y la coherencia entre las operaciones visibles y los datos observados en el panel de Supabase.

Las siguientes secciones separan los grupos de pruebas más relevantes para evitar mezclar autenticación, persistencia, cámara y compatibilidad de interfaz.

13.4. Pruebas de autenticación

Se verificaron registro por correo, confirmación, acceso con credenciales válidas e inválidas, persistencia de sesión, cierre de sesión, recuperación de contraseña mediante deep link, actualización de contraseña, Google OAuth y eliminación de cuenta.

La eliminación se contrastó directamente en Supabase Authentication. Tras borrar una cuenta y autenticarse posteriormente con Google usando el mismo correo, Supabase crea correctamente una nueva identidad, comportamiento esperado de OAuth cuando la cuenta anterior ya no existe.

Prueba funcional	Resultado esperado	Estado
Registro por correo	Alta en Supabase y tratamiento de confirmación.	Correcto
Login por correo	Acceso con credenciales válidas y error claro con inválidas.	Correcto
Persistencia de sesión	Reapertura con sesión válida sin autenticación repetida.	Correcto
Recuperación de contraseña	Deep link abre el flujo de cambio de contraseña.	Correcto
Google OAuth	Autenticación externa y retorno a la aplicación.	Correcto
Cerrar sesión	Finalizar sesión y limpiar estado de usuario.	Correcto
Eliminar cuenta	Usuario eliminado de Supabase Authentication.	Correcto
Catálogo y búsqueda	Datos remotos visibles y filtros/búsqueda coherentes.	Correcto
Favoritos	Alta/baja y persistencia por usuario.	Correcto
Carrito	Añadir, modificar, eliminar, vaciar y restaurar cantidades.	Correcto
AlgaeChef	Opciones rápidas/texto generan recomendación enlazada a receta real.	Correcto
Tienda oficial	Apertura del enlace externo correspondiente.	Correcto

Tabla 30. Resumen de pruebas funcionales de los flujos principales.

13.5. Pruebas de persistencia multiusuario

Favoritos y carrito se validaron cerrando y restaurando sesiones y comprobando que los datos recuperados correspondían al usuario autenticado. Las tablas privadas incluyen user_id y políticas RLS, por lo que las consultas no dependen únicamente de filtros de interfaz.

También se probaron altas, bajas, vaciado de favoritos, modificación rápida de cantidades, eliminación de productos del carrito y vaciado completo. La interfaz refleja los cambios inmediatamente y, después de la sincronización, el estado persiste en Supabase.

13.6. Pruebas del QR

La lectura QR se probó con productos físicos y con códigos de ensayo generados para referencias del catálogo sin un QR físico disponible. Los códigos reales de Furikake y Kombu contienen URLs de Porto-Muiños, mientras que los códigos de prueba permitieron cubrir identificadores y formatos internos.

En total se validó la resolución de diez productos del conjunto de prueba. En el emulador se utilizó la cámara virtual/webcam y, posteriormente, el flujo se repitió en dispositivo físico. En ambos casos, una detección válida termina en la ficha interna correspondiente.

13.7. Pruebas en emulador

El emulador principal utilizado fue un Pixel 9 Pro XL con API 36. Permitió comprobar navegación, tamaños, estados de autenticación, persistencia y gran parte de la interacción sin depender continuamente de un dispositivo físico.

También se utilizó para reproducir configuraciones alternativas del sistema, como navegación por gestos y navegación clásica de tres botones, y para probar el escáner QR mediante la cámara virtual.

13.8. Pruebas en dispositivo físico

Se generó una APK de depuración y se instaló directamente en un teléfono Android. La aplicación funcionó correctamente fuera del entorno de Android Studio, incluyendo conexión con Supabase, carga de imágenes, autenticación, Google OAuth, QR con cámara real, favoritos, carrito, enlaces externos y AlgaeChef.

La prueba física resultó especialmente importante porque permitió detectar diferencias de insets y navegación del sistema que no eran visibles con la configuración inicial del emulador.

13.9. Pruebas de interfaz y modos de navegación

Con navegación por gestos, la barra inferior flotante se mostraba correctamente. Al utilizar la navegación clásica de tres botones en un dispositivo real, la barra del sistema ocupaba más altura y algunos elementos inferiores podían quedar demasiado próximos o parcialmente cubiertos.

La solución consistió en calcular el espacio inferior a partir de WindowInsets.navigationBars y utilizar navigationBarsPadding() donde correspondía. Después se reprodujo la configuración de tres botones también en el emulador y se verificaron Tienda, detalle de producto, carrito y login. El mismo diseño continúa funcionando con gestos.

13.10. Inspección estática

Inspect Code se ejecutó sobre el proyecto completo como comprobación adicional. La herramienta no mostró errores que impidieran compilar o ejecutar la aplicación; sí generó advertencias y recomendaciones de distinta prioridad.

Se revisaron especialmente declaraciones no utilizadas, imports, versiones de librerías, recursos, rendimiento de vectores, recomendaciones KTX, compatibilidad de imágenes y comprobaciones de internacionalización. Las sugerencias sin impacto funcional o que implicaban una migración de dependencias al final del proyecto se documentaron y se dejaron fuera del alcance.

13.11. Resultados

El conjunto de pruebas permitió cerrar la versión final con compilación correcta, las 13 pruebas unitarias superadas y los flujos principales verificados en emulador y dispositivo físico. No se detectaron errores bloqueantes pendientes.

Entorno / comprobación	Resultado
Gradle + JDK 21	testDebugUnitTest: BUILD SUCCESSFUL.
Emulador Pixel 9 Pro XL API 36	Flujos principales y QR mediante cámara virtual correctos.
Dispositivo Android físico	APK instalada y funcionalidades principales correctas.
Navegación por gestos	Bottom bar e insets correctos.
Navegación de tres botones	Ajuste de insets verificado tras la corrección.
Inspect Code	Sin errores bloqueantes; advertencias no críticas revisadas.

Tabla 31. Resultado de la validación final por entorno y herramienta.

La validación no pretende demostrar ausencia absoluta de defectos. Funcionalidades externas como disponibilidad de Supabase, Google OAuth o la tienda oficial dependen de servicios de terceros, y una futura publicación comercial requeriría ampliar la automatización con pruebas instrumentadas, integración continua y monitorización.

14. Problemas encontrados y soluciones adoptadas

14.1. Configuración de Supabase

La integración inicial requirió coordinar la versión del SDK de Supabase, los módulos PostgREST, Auth y Functions y el cliente HTTP de Ktor. La configuración quedó

centralizada en SupabaseClientProvider y las credenciales se trasladaron a local.properties para evitar valores sensibles embebidos en el código.

También se documentó el backend dentro del proyecto mediante supabase/schema.sql y la Edge Function delete-account, de modo que la estructura necesaria para reproducir la solución no dependa exclusivamente del panel web.

14.2. Autenticación y deep links

La recuperación de contraseña no podía resolverse únicamente enviando un correo: era necesario que el enlace devuelto por Supabase reabriera la aplicación y permitiera distinguir una recuperación de un login externo. Se configuró el esquema portomuinos://auth en Supabase y AndroidManifest y se añadió el procesamiento de deep links en MainActivity.

AuthDeepLinkManager actúa como punto intermedio y expone dos estados: recuperación de contraseña pendiente y autenticación externa pendiente. La navegación consume el evento correspondiente y evita interpretar ambos retornos de la misma manera.

Durante las pruebas también se alcanzó temporalmente el límite de envío de correos del servicio SMTP incluido en Supabase. Se trató como una limitación del entorno de prueba y se incorporó un mensaje específico para el usuario; una puesta en producción requeriría revisar el proveedor de correo transaccional.

14.3. Google OAuth

Google OAuth requirió configurar un proyecto en Google Auth Platform, crear un cliente web, registrar el callback de Supabase y activar el proveedor Google en el backend. Posteriormente se configuró el deep link móvil para recuperar el control tras el navegador.

Una particularidad del flujo es que «Continuar con Google» funciona tanto como acceso como registro. Si la identidad no existe, Supabase puede crearla después de que Google la autentique. Por ese motivo no se fuerza al usuario a pasar primero por la pantalla de registro tradicional.

14.4. Lectura QR

Los primeros ensayos mostraron que los códigos físicos no seguían necesariamente un identificador interno de la aplicación: algunos productos de Porto-Muiños contienen URLs oficiales. Limitar el escáner a product_1, product_2, etc. habría hecho incompatible el prototipo con envases reales.

La solución fue crear QrProductResolver, que genera candidatos a partir del valor completo, último segmento de ruta y varios parámetros habituales. Después compara esos candidatos con id y qr_code de los productos cargados. Así se admite tanto un QR de ensayo como una URL oficial sin introducir lógica específica en la pantalla de cámara.

14.5. Persistencia de favoritos y carrito

Favoritos y carrito comenzaron como estados locales suficientes para validar la interfaz, pero no cumplían el comportamiento esperado de una cuenta real. Se crearon tablas específicas en Supabase, claves foráneas hacia auth.users y políticas RLS para que cada usuario acceda únicamente a sus datos.

Después se trasladó la lógica a FavoriteRepository y CartRepository y se añadió una carga inicial tras la autenticación. Este cambio permitió conservar los datos entre reinicios y separar correctamente la información de distintos usuarios.

14.6. Sincronización rápida del carrito

Una interfaz optimista presenta un problema cuando el usuario modifica una cantidad varias veces antes de que terminen las peticiones anteriores. Si las respuestas remotas finalizan en un orden diferente al de las pulsaciones, una cantidad antigua podría sobrescribir a la última.

CartViewModel evita este caso encadenando las operaciones mediante un Job: cada escritura espera la finalización de la anterior. FavoritesViewModel utiliza la misma idea por identificador de producto o receta. En caso de fallo se restaura o vuelve a consultar el estado remoto solo cuando no existe una acción más reciente que deba prevalecer.

14.7. Adaptación a navegación por gestos y tres botones

La aplicación se diseñó inicialmente con navegación gestual, donde el inset inferior del sistema es reducido. En un teléfono configurado con los tres botones clásicos, Android eleva el contenido disponible y la barra inferior de la aplicación podía acercarse o solaparse con elementos de algunas pantallas.

La corrección consistió en abandonar una reserva inferior completamente fija y sumar el inset real de WindowInsets.navigationBars. El mensaje temporal del login recibió el mismo tratamiento. La solución se verificó después en ambas configuraciones sin rediseñar individualmente cada pantalla.

14.8. Crecimiento de PortoMuinosApp.kt

A medida que se incorporaron autenticación, catálogo, favoritos, carrito y AlgaeChef, PortoMuinosApp acumuló estados y operaciones que hacían más difícil identificar responsabilidades. Aunque el comportamiento era correcto, el archivo se estaba convirtiendo en un punto de coordinación excesivamente amplio.

La solución no fue dividir código de forma arbitraria, sino extraer responsabilidades con entidad propia. Los estados de negocio pasaron a ViewModels y el grafo de pantallas a PortoNavHost. La raíz conserva únicamente coordinación global, dependencias de Compose y estados estrictamente vinculados al nivel de aplicación.

14.9. Refactorización a MVVM

La migración a MVVM se abordó cuando la aplicación ya disponía de los flujos principales, lo que reducía el riesgo de diseñar una arquitectura teórica alejada de las necesidades reales. Cada ViewModel se introdujo manteniendo el contrato de las pantallas y comprobando después que no cambiaba el resultado funcional.

No se incorporaron capas adicionales como Use Cases, Domain, Hilt, Room o una Clean Architecture estricta porque no existía una responsabilidad concreta que justificase su coste en este alcance. La decisión final busca equilibrio entre separación, comprensión y tamaño del proyecto.

14.10. Gestión del alcance de AlgaeChef

La propuesta conceptual de AlgaeChef contemplaba evolucionar hacia recomendaciones generadas con inteligencia artificial. Integrar un modelo externo en la fase final habría añadido selección de proveedor, gestión segura de claves, costes de API, diseño de prompts, control de respuestas, tratamiento de errores y validación de recomendaciones.

Se priorizó por ello un demostrador determinista que reproduce el recorrido de usuario y devuelve recetas reales. Esta decisión permite evaluar la experiencia completa sin presentar como terminada una IA que no ha sido validada. La separación en AlgaeChefViewModel y en una función generadora independiente deja además un punto claro de sustitución para una versión futura.

Problema detectado	Solución adoptada	Resultado
Configuración y credenciales de Supabase	Provider central + local.properties + esquema reproducible.	Backend desacoplado y configuración documentada.
Deep links de Auth	Esquema portomuinos://auth + gestor de eventos.	Recuperación y OAuth regresan al flujo correcto.
QR físicos con URLs	Normalización flexible en QrProductResolver.	Compatibilidad con códigos reales y de prueba.
Favoritos/carrito locales	Tablas por usuario + Repository + RLS.	Persistencia real y aislamiento por cuenta.
Cambios rápidos de cantidad	Sincronización secuencial y rollback.	Se conserva el orden de las acciones.
Navegación Android de tres botones	WindowInsets dinámicos.	UI válida con gestos y tres botones.
Acumulación de lógica en raíz Compose	Refactorización a ViewModels y PortoNavHost.	Responsabilidades más claras.
AlgaeChef con IA real fuera de plazo	Demostrador determinista con recetas reales.	Flujo funcional sin sobreafirmar capacidades.

Tabla 32. Síntesis de problemas relevantes y soluciones adoptadas.

15. Innovación y aspectos destacables

La innovación del proyecto no se plantea como la creación aislada de una tecnología inexistente, sino como la combinación de capacidades conocidas dentro de una experiencia especializada para Porto-Muiños. El valor diferencial aparece al relacionar contenido gastronómico, producto físico, personalización y acceso a compra en una

misma aplicación móvil, manteniendo además una arquitectura y un backend suficientemente preparados para evolucionar más allá del prototipo académico.

15.1. Integración del ecosistema producto-receta-compra

Uno de los aspectos más destacables es la continuidad entre los distintos tipos de contenido. El catálogo no funciona como una colección independiente de productos y el recetario no se limita a mostrar preparaciones: ambos se conectan mediante recipe_products. Esto permite que una receta conduzca a los productos Porto-Muiños utilizados y que una ficha de producto muestre recetas en las que puede emplearse. El carrito y los enlaces oficiales completan el recorrido hasta el canal comercial existente, sin duplicar un sistema de pago dentro de la aplicación.

15.2. QR como puente físico-digital

El escaneo QR incorpora una conexión directa entre el envase físico y la experiencia digital. En lugar de utilizar el código únicamente como enlace web, la aplicación interpreta el contenido y, cuando identifica una referencia compatible, abre la ficha interna correspondiente. El usuario mantiene así el contexto de la app y puede continuar hacia recetas relacionadas, favoritos o compra. La resolución flexible de identificadores y URLs permite además trabajar tanto con códigos de ensayo como con códigos presentes en productos reales.

15.3. Personalización por usuario

Favoritos y carrito dejan de ser estados temporales de la interfaz y se convierten en información vinculada a la cuenta autenticada. El uso de tablas específicas, claves foráneas hacia auth.users y políticas RLS permite recuperar esa información después de cerrar la aplicación y, al mismo tiempo, aislarla entre usuarios. La personalización implementada es deliberadamente sencilla, pero establece una base real para ampliar posteriormente preferencias, historial o recomendaciones sin rediseñar la autenticación ni el modelo de seguridad.

15.4. AlgaeChef como concepto de asistente culinario

AlgaeChef introduce una forma de interacción diferente al buscador tradicional. El usuario expresa una intención —por texto libre o mediante opciones rápidas— y recibe

una recomendación contextual que desemboca en una receta existente. En la versión entregada el generador es determinista y no se presenta como inteligencia artificial generativa; la innovación reside en haber validado la experiencia, la integración con el catálogo y el punto de extensión técnico antes de asumir el coste y la incertidumbre de un modelo externo.

15.5. Integración de tecnologías cloud en una aplicación Android

La solución combina un cliente Android nativo con servicios cloud gestionados: PostgreSQL y PostgREST para datos, Supabase Auth para identidad, Storage para imágenes y una Edge Function para una operación administrativa que no debe ejecutarse desde el APK. Esta integración permite mantener en el servidor la persistencia, la integridad y la autorización, mientras que la aplicación se concentra en la experiencia móvil. Para el alcance del proyecto, este planteamiento aporta capacidades propias de una aplicación conectada sin necesidad de desplegar y mantener una API REST y un servidor dedicados.

Aspecto	Aplicación en el proyecto	Valor aportado
Producto-receta-compra	Navegación bidireccional y continuidad desde la inspiración hasta el canal comercial.	Reduce la fragmentación entre contenido gastronómico y producto.
QR físico-digital	Resolución de códigos reales y de prueba hacia fichas internas.	Convierte el envase en punto de entrada a recetas e información contextual.
Estado personal	Favoritos y carrito persistentes protegidos por RLS.	La experiencia se conserva por usuario y no solo durante una sesión visual.
AlgaeChef	Interfaz conversacional validada con recomendaciones deterministas y recetas reales.	Prepara una futura evolución hacia IA sin depender todavía de un modelo externo.
Backend cloud	Auth, PostgreSQL, Storage, PostgREST y Functions integrados con Android.	Permite persistencia y seguridad reales manteniendo un alcance de infraestructura asumible.

Tabla 33. Síntesis de los aspectos innovadores y diferenciales del proyecto.

16. Conclusiones

El desarrollo de Porto-Muiños App permite cerrar el proyecto con una aplicación Android funcional, conectada a servicios reales y coherente con el problema planteado al inicio

de la memoria. Las conclusiones se formulan a partir del resultado implementado y de las pruebas realizadas, diferenciando las capacidades validadas de las ampliaciones que permanecen fuera del alcance.

16.1. Resultados obtenidos

El resultado principal es una aplicación nativa desarrollada con Kotlin y Jetpack Compose que integra autenticación, catálogo, recetas, búsqueda, relaciones producto-receta, favoritos, carrito, QR, perfil, conexión con la tienda oficial y un demostrador de AlgaeChef. Supabase proporciona persistencia remota, autenticación, almacenamiento y seguridad por filas, mientras que las pruebas realizadas confirman el funcionamiento de los flujos principales tanto en emulador como en un dispositivo físico.

La validación final incluye 13 pruebas unitarias superadas, pruebas funcionales sobre servicios reales, lectura de QR, persistencia multiusuario, autenticación por correo y Google, ejecución mediante APK y revisión estática con Android Studio. Estas evidencias no eliminan la posibilidad de defectos futuros, pero permiten afirmar que la versión entregada cumple el comportamiento previsto para el alcance definido.

16.2. Cumplimiento de objetivos

Los objetivos establecidos en el capítulo 2 se consideran cumplidos dentro de los límites declarados. La aplicación Android nativa, el backend remoto, la autenticación, el QR, la personalización básica, la arquitectura MVVM, la seguridad mediante RLS y la validación están presentes en la versión final. AlgaeChef cumple su objetivo como demostrador funcional, no como sistema de IA generativa. Del mismo modo, Apple, preferencias alimentarias avanzadas y checkout propio se mantienen expresamente fuera del alcance y no se contabilizan como objetivos incumplidos.

16.3. Aprendizajes técnicos

El proyecto ha permitido integrar conocimientos de distintas áreas del ciclo en una solución única: desarrollo Android con Kotlin y Compose, programación asíncrona y gestión de estado, modelado relacional, autenticación y OAuth, seguridad mediante RLS, consumo de servicios remotos, almacenamiento de recursos, acceso a cámara y

reconocimiento QR, navegación mediante deep links, pruebas y refactorización arquitectónica.

Uno de los aprendizajes más relevantes ha sido comprobar que la calidad no depende únicamente de que una pantalla funcione. Decisiones como trasladar favoritos y carrito al backend, introducir claves y restricciones adecuadas, separar responsabilidades mediante ViewModels y repositories o adaptar la interfaz a los insets reales del sistema mejoran aspectos menos visibles, pero fundamentales para la mantenibilidad y la consistencia de una aplicación conectada.

16.4. Valoración personal del proyecto

A nivel personal, el proyecto ha supuesto transformar una idea inicialmente conceptual en un producto que puede instalarse, utilizarse y comprobarse fuera del entorno de desarrollo. El proceso ha exigido tomar decisiones de alcance, descartar soluciones que no aportaban suficiente valor en la fase disponible y revisar otras cuando las pruebas reales mostraron limitaciones. Esa evolución ha resultado especialmente útil para comprender el desarrollo como un proceso iterativo y no como una sucesión lineal de pantallas.

La versión final no pretende presentarse como un producto comercial cerrado. Su valor académico reside en haber alcanzado un conjunto funcional coherente, con backend, seguridad, persistencia, integraciones y validación reales, y en dejar identificados de forma explícita los puntos que requerirían trabajo adicional antes de una publicación de producción.

17. Líneas futuras y propuestas de mejora

Las siguientes líneas no forman parte de la versión evaluada, pero derivan de necesidades detectadas durante el desarrollo y de elementos ya previstos en la interfaz. Se plantean como evoluciones graduales sobre la arquitectura actual, priorizando aquellas que aportan valor sin comprometer seguridad, mantenibilidad o claridad para el usuario.

17.1. AlgaeChef con IA generativa real

La evolución más directa de AlgaeChef sería sustituir el generador determinista por un modelo generativo capaz de interpretar ingredientes disponibles, preferencias y objetivos culinarios. La integración debería realizarse desde backend o una función intermedia para no distribuir claves privadas en el APK. Las respuestas deberían apoyarse en datos del catálogo y devolver una estructura controlada —por ejemplo, producto recomendado, explicación y recipe_id— que permita mantener la navegación hacia contenido real.

Antes de una publicación sería necesario definir proveedor, costes, límites de uso, tratamiento de errores, moderación y criterios de calidad. La IA no debería inventar productos inexistentes ni presentar recomendaciones sensibles como garantías; su función sería complementar el conocimiento gastronómico de la aplicación y no sustituir la información oficial de Porto-Muiños.

17.2. Preferencias alimentarias

La pantalla de perfil ya reserva espacio para preferencias alimentarias. Una ampliación permitiría almacenar opciones como vegetariano, vegano o tipos de cocina preferidos y utilizarlas para ordenar o filtrar recetas y recomendaciones. Para que la configuración acompañase al usuario entre dispositivos, estos datos deberían persistirse en el backend asociados a auth.users, en lugar de depender exclusivamente de preferencias locales.

17.3. Alergias e intolerancias

Las alergias e intolerancias requieren un tratamiento más estricto que una preferencia de sabor. Una futura implementación podría permitir declarar alérgenos y advertir cuando una receta contenga ingredientes potencialmente incompatibles, pero no debería presentarse como una garantía médica automática. Sería necesario disponer de datos de ingredientes suficientemente fiables, mostrar advertencias claras y remitir siempre al etiquetado oficial del producto cuando exista cualquier duda.

17.4. Autenticación Apple

La interfaz conserva la opción «Continuar con Apple» como funcionalidad futura. Su implementación requeriría configurar el proveedor correspondiente, las credenciales y callbacks del ecosistema Apple y verificar el retorno mediante el mismo flujo de autenticación utilizado por la aplicación. El objetivo sería mantener una experiencia equivalente a Google sin crear un sistema de identidad independiente.

17.5. Contraseña para cuentas OAuth

Una cuenta creada exclusivamente mediante Google no dispone de una contraseña local que la aplicación pueda conocer. Como mejora se podría incorporar un flujo específico para añadir un método de acceso por correo y contraseña o vincular identidades, siempre utilizando las capacidades admitidas por Supabase Auth. Esta opción debería diseñarse de forma explícita desde ajustes de cuenta, evitando sugerir que la contraseña de Google puede reutilizarse dentro de Porto-Muiños App.

17.6. Perfil sincronizado en backend

Actualmente el nombre se obtiene de los metadatos de Auth y la imagen de perfil se conserva localmente. Una evolución natural sería crear una entidad de perfil asociada al usuario y almacenar el avatar en un espacio privado o controlado de Storage. Esto permitiría mantener imagen, preferencias y otros datos de personalización al iniciar sesión desde otro dispositivo, aplicando políticas RLS equivalentes a las utilizadas en favoritos y carrito.

17.7. Compra integrada

El carrito actual deriva el pago y el envío a la tienda oficial, decisión adecuada para el alcance desarrollado. Si Porto-Muiños necesitase en el futuro una compra totalmente integrada, habría que definir catálogo comercial sincronizado, direcciones, impuestos, transporte, pagos, confirmaciones y tratamiento de errores. La incorporación de un proveedor de pago debería realizarse siguiendo sus mecanismos seguros y sin almacenar datos sensibles de tarjetas en la aplicación.

17.8. Notificaciones

Las notificaciones podrían utilizarse para comunicar nuevas recetas, contenido estacional o novedades de producto, siempre con consentimiento del usuario y posibilidad de desactivarlas. Una implementación habitual en Android podría apoyarse en un servicio de mensajería push y en preferencias almacenadas por cuenta para evitar comunicaciones indiscriminadas. Su utilidad debería validarse antes de convertirlas en una fuente de interrupciones.

17.9. Internacionalización

La aplicación se ha desarrollado en español. Una versión comercial podría incorporar gallego, inglés u otros idiomas, trasladando los textos de interfaz a recursos localizables y definiendo también una estrategia para el contenido remoto de productos y recetas. Internacionalizar no consiste únicamente en traducir botones: deben revisarse formatos, textos del backend, enlaces y contenido editorial para mantener una experiencia coherente en cada idioma.

17.10. Panel de administración y gestión de contenidos

El catálogo se mantiene actualmente desde Supabase. Para un uso continuado por parte de la empresa sería conveniente disponer de una herramienta de gestión orientada a personal no técnico, desde la que crear y editar productos, recetas, relaciones, imágenes, códigos y enlaces comerciales. Este panel debería incorporar autenticación administrativa y permisos diferenciados, manteniendo separado el acceso interno de las operaciones disponibles para usuarios de la aplicación.

Horizonte orientativo	Líneas principales	Motivo de la prioridad
Corto plazo	Autenticación Apple, perfil sincronizado y mejora de localización de textos.	Amplían comodidad y consistencia sin alterar el núcleo funcional.
Medio plazo	Preferencias, alergias con tratamiento seguro, notificaciones y AlgaeChef generativo.	Requieren nuevos datos, reglas de negocio o servicios externos y una validación adicional.
Largo plazo	Compra integrada y panel de administración completo.	Implican procesos empresariales, seguridad, mantenimiento y coordinación con la infraestructura comercial.

Tabla 34. Priorización orientativa de las principales líneas de evolución.

La prioridad final de estas mejoras debería decidirse a partir de uso real, feedback de usuarios y objetivos de Porto-Muiños. La arquitectura actual permite abordar varias de ellas de forma incremental, pero una puesta en producción comercial requeriría volver a estimar alcance, costes, mantenimiento y pruebas antes de incorporarlas a una versión pública.

18. Bibliografía y webgrafía

Fuentes consultadas para fundamentar el estudio de mercado y las decisiones técnicas descritas en la memoria. Última consulta: 30 de agosto de 2026.

Mercado, producto y competencia

- [1] Porto-Muiños. Web oficial, recetas y tienda online. https://www.portomuinos.com/
- [2] SideChef. Shop and Cook Step-by-Step Recipes. https://www.sidechef.com/
- [3] Samsung Food. The Ultimate Cooking App. https://samsungfood.com/
- [4] Kitchen Stories. Recipes. https://www.kitchenstories.com/en/recipes

Documentación técnica oficial

- [5] Android Developers. Guide to app architecture. https://developer.android.com/topic/architecture
- [6] Android Developers. ViewModel overview. https://developer.android.com/topic/libraries/architecture/viewmodel

- [7] Android Developers. Jetpack Compose documentation. https://developer.android.com/develop/ui/compose/documentation
- [8] Android Developers. Navigation. https://developer.android.com/guide/navigation
- [9] JetBrains. Kotlin Documentation. https://kotlinlang.org/docs/home.html
- [10] Gradle. Gradle User Manual. https://docs.gradle.org/current/userguide/
- [11] Supabase. Kotlin Client Reference. https://supabase.com/docs/reference/kotlin/introduction
- [12] Supabase. Auth. https://supabase.com/docs/guides/auth
- [13] Supabase. Login with Google. https://supabase.com/docs/guides/auth/social-login/auth-google
- [14] Supabase. Row Level Security. https://supabase.com/docs/guides/database/postgres/row-level-security
- [15] Supabase. Storage. https://supabase.com/docs/guides/storage
- [16] Supabase. Edge Functions. https://supabase.com/docs/guides/functions
- [17] PostgreSQL Global Development Group. PostgreSQL Documentation. https://www.postgresql.org/docs/
- [18] Android Developers. CameraX overview. https://developer.android.com/media/camera/camerax
- [19] Google for Developers. Scan barcodes with ML Kit on Android. https://developers.google.com/ml-kit/vision/barcode-scanning/android
- [20] Coil. Compose. https://coil-kt.github.io/coil/compose/
- [21] JetBrains. Kotlin Serialization. https://kotlinlang.org/docs/serialization.html
- [22] JetBrains. Ktor Documentation. https://ktor.io/docs/welcome.html

Anexo A. Manual técnico e instalación

A.1. Finalidad del manual

El presente anexo describe los requisitos y pasos necesarios para instalar, configurar y ejecutar Porto-Muiños App, tanto a partir del proyecto Android como mediante una APK previamente generada. También se documentan los servicios externos necesarios para reproducir el entorno utilizado durante el desarrollo.

La aplicación está formada por un cliente Android nativo conectado a Supabase. El cliente contiene la interfaz y la lógica de presentación, mientras que Supabase proporciona autenticación, base de datos PostgreSQL, almacenamiento de imágenes, APIs y la función serverless empleada para la eliminación de cuentas. La aplicación necesita conexión a Internet para acceder a estos servicios y para abrir los contenidos de la tienda oficial de Porto-Muiños.

A.2. Requisitos del entorno

Para ejecutar el proyecto desde su código fuente se necesita un equipo compatible con Android Studio y los siguientes elementos:

Recurso	Configuración utilizada
IDE	Android Studio
JDK para Gradle	JDK 21
Gradle	8.13
Lenguaje	Kotlin
Interfaz	Jetpack Compose
compileSdk	36
targetSdk	36
minSdk	26
Objetivo JVM del módulo Android	JVM 11
Backend	Supabase
Dispositivo mínimo	Android 8.0 / API 26
Conexión	Internet
Hardware adicional	Cámara para escaneo QR

Tabla 35. Requisitos técnicos y configuración utilizada para ejecutar el proyecto.

Durante la validación final se utilizó JDK 21 para ejecutar Gradle 8.13 y las pruebas unitarias. El módulo Android mantiene un objetivo JVM 11, compileSdk y targetSdk 36 y minSdk 26. La aplicación fue comprobada tanto en un emulador Pixel 9 Pro XL con API 36 como en un dispositivo Android físico.

A.3. Apertura y configuración del proyecto Android

Una vez disponible el código fuente:

1. Abrir Android Studio.
2. Seleccionar Open y escoger el directorio raíz del proyecto.
3. Esperar a que Android Studio complete la sincronización de Gradle y descargue las dependencias necesarias.
4. Comprobar que el JDK empleado para Gradle es JDK 21.
5. Verificar que se encuentra instalado el SDK de Android necesario para compilar el proyecto.
6. Configurar las credenciales de Supabase antes de ejecutar la aplicación.

Las credenciales no se encuentran codificadas directamente en el código fuente. El proyecto utiliza local.properties y expone posteriormente los valores necesarios mediante BuildConfig. El archivo real de configuración no debe añadirse al repositorio. Para facilitar la reproducción se incluye local.properties.example, que contiene los nombres de las variables requeridas sin valores privados.

La configuración utilizada por la aplicación requiere los valores correspondientes a la URL del proyecto Supabase y a la clave pública de cliente:

```
SUPABASE_URL=<URL_DEL_PROYECTO_SUPABASE>
SUPABASE_KEY=<CLAVE_PUBLICA_DEL_PROYECTO>
```

No debe incorporarse al cliente Android ninguna clave service_role ni otra credencial administrativa.

A.4. Configuración del backend Supabase

Para reproducir completamente la aplicación es necesario disponer de un proyecto Supabase.

A.4.1. Base de datos

El proyecto incluye el archivo supabase/schema.sql. Este script documenta la estructura persistente utilizada por la aplicación y permite reproducir las tablas, relaciones y políticas necesarias del backend.

El esquema comprende:

- products
- recipes
- recipe_products
- favorite_products
- favorite_recipes
- cart_items

Las cuentas de usuario son gestionadas directamente mediante auth.users de Supabase Auth. Favoritos y carrito se relacionan con estos identificadores y utilizan políticas Row Level Security para impedir que una cuenta pueda acceder a los datos privados de otra.

Tras crear un proyecto Supabase, el esquema SQL debe ejecutarse sobre su base de datos antes de utilizar las funcionalidades dependientes de persistencia.

A.4.2. Autenticación

Supabase Auth debe estar habilitado para:

- registro mediante correo y contraseña;
- confirmación de correo;
- inicio de sesión;
- recuperación de contraseña;
- persistencia de sesión;
- Google OAuth.

La recuperación de contraseña y el retorno de Google OAuth utilizan el deep link:

```
portomuinos://auth
```

El mismo esquema se encuentra declarado en Android y debe estar permitido como URL de redirección en la configuración de autenticación de Supabase.

A.4.3. Google OAuth

Para habilitar el botón «Continuar con Google» es necesario configurar previamente el proveedor Google en Supabase.

La configuración utilizada durante el proyecto requirió:

1. crear o utilizar un proyecto en Google Auth Platform;

2. configurar un cliente OAuth web;
3. registrar el callback proporcionado por Supabase;
4. introducir las credenciales correspondientes en el proveedor Google de Supabase;
5. activar el proveedor;
6. mantener configurado portomuinos://auth para devolver posteriormente el control a la aplicación.

La autenticación se realiza entre Google y Supabase. Porto-Muiños App no recibe ni almacena la contraseña de la cuenta Google del usuario.

A.4.4. Storage

Las imágenes remotas del catálogo se almacenan en el bucket público catalog-media. La configuración utilizada admite imágenes JPEG, PNG y WebP, con un tamaño máximo de 5 MiB por archivo. La aplicación puede trabajar tanto con una URL completa como con una ruta interna del bucket, construyendo en este último caso la URL pública a partir de SUPABASE_URL.

A.4.5. Función delete-account

La eliminación definitiva de una cuenta no se ejecuta directamente desde el APK, ya que requiere privilegios administrativos. Para esta operación se utiliza la Edge Function delete-account. La función recibe una petición autenticada, identifica al usuario actual y ejecuta la eliminación desde el entorno servidor. Las credenciales administrativas permanecen así fuera de la aplicación Android.

Para reproducir esta funcionalidad debe desplegarse en el proyecto Supabase la función incluida con el código fuente y configurarse en el entorno servidor con los permisos necesarios.

A.5. Ejecución desde Android Studio

Una vez configurados Android y Supabase:

1. iniciar un emulador Android compatible o conectar un dispositivo físico;
2. si se utiliza un teléfono físico, habilitar las opciones necesarias para depuración mediante USB;
3. seleccionar el dispositivo desde Android Studio;

4. ejecutar la configuración principal de la aplicación;
5. esperar a la compilación e instalación.

La pantalla Splash inicializa el estado de autenticación y determina si el usuario debe acceder al flujo de inicio de sesión o puede continuar directamente a Home.

Para comprobar el lector QR debe concederse permiso de cámara cuando Android lo solicite. Las pruebas realizadas durante el proyecto confirmaron su funcionamiento tanto mediante la cámara virtual del emulador como utilizando la cámara de un dispositivo físico.

A.6. Instalación mediante APK

La aplicación también puede utilizarse sin Android Studio mediante la APK generada durante el proyecto.

El procedimiento general es:

1. copiar la APK al dispositivo Android;
2. abrir el archivo desde el dispositivo;
3. autorizar temporalmente la instalación desde esa fuente si Android lo solicita;
4. confirmar la instalación;
5. abrir Porto-Muiños App desde el launcher.

El dispositivo debe utilizar Android 8.0 o superior y disponer de conexión a Internet. Para utilizar la función de escaneo QR deberá contar además con cámara y conceder el correspondiente permiso.

La APK release final firmada, PortoMuinos-v1.0.apk, fue instalada y validada correctamente en un dispositivo físico antes de su publicación.

A.7. Comprobación de la instalación

Después de la primera ejecución se recomienda realizar una comprobación mínima del entorno:

Comprobación	Resultado esperado
Inicio de la app	Splash y navegación al acceso o Home
Registro por correo	Creación de cuenta en Supabase
Inicio de sesión	Acceso correcto a la aplicación
Google OAuth	Retorno a la app tras autenticación
Catálogo	Productos cargados desde Supabase
Recetario	Recetas y relaciones disponibles
Favoritos	Persistencia asociada al usuario
Carrito	Cantidades conservadas tras reiniciar
QR	Apertura de la ficha correspondiente
Compra	Apertura del contenido oficial de Porto-Muiños
Cierre de sesión	Eliminación de la sesión activa

Tabla 36. Comprobaciones básicas posteriores a la instalación.

Estas comprobaciones coinciden con los flujos principales validados durante la fase final del proyecto. La validación incluyó además 13 pruebas unitarias, pruebas funcionales, emulador, dispositivo físico e inspección estática.

A.8. Incidencias habituales

La aplicación compila incorrectamente o Gradle muestra incompatibilidades. Debe comprobarse que Gradle utiliza JDK 21 y que el SDK de Android requerido está instalado.

La aplicación abre pero no carga catálogo ni recetas. Debe revisarse la conexión a Internet y la configuración de Supabase almacenada en local.properties.

El usuario puede iniciar sesión pero favoritos o carrito no funcionan correctamente. Debe comprobarse que las tablas del backend y sus políticas RLS se han creado correctamente.

La recuperación de contraseña no vuelve a la aplicación. Debe comprobarse que portomuinos://auth está configurado tanto en Supabase como en la aplicación Android.

Google OAuth no devuelve el control a la app. Debe revisarse la configuración del proveedor Google, el callback de Supabase y el deep link móvil.

Las imágenes del catálogo no aparecen. Debe comprobarse la disponibilidad del bucket catalog-media, las rutas almacenadas y el acceso público a los recursos.

El escáner QR no muestra la cámara. Debe comprobarse que el permiso de cámara ha sido concedido.

No se recibe un correo de registro o recuperación. El servicio SMTP incluido en el entorno de prueba de Supabase dispone de límites de envío. Durante el desarrollo se alcanzó temporalmente este límite, por lo que una implantación comercial debería configurar un proveedor de correo transaccional adecuado.

A.9. Consideraciones de seguridad

El procedimiento de instalación y configuración mantiene la separación entre credenciales públicas de cliente y secretos administrativos. local.properties no debe incluirse en el repositorio. La aplicación utiliza únicamente la clave pública necesaria para conectarse a Supabase y la capacidad real de acceso a los datos se restringe mediante Row Level Security. Las operaciones que requieren privilegios administrativos se ejecutan desde backend; en particular, la eliminación de cuentas se delega en la Edge Function delete-account.

Anexo B. Manual de usuario

Este manual describe las operaciones principales que puede realizar una persona usuaria en Porto-Muiños App. Las pantallas mostradas corresponden a la versión final validada en dispositivo físico y permiten recorrer los flujos de acceso, exploración de recetas y productos, escaneo QR, personalización, carrito y gestión de cuenta.

B.1. Acceso, registro y sesión

Cuando no existe una sesión activa, la aplicación muestra la pantalla de inicio de sesión. El usuario puede acceder con correo electrónico y contraseña, utilizar Google OAuth o abrir el formulario de creación de cuenta. Para registrarse mediante correo es necesario indicar nombre completo, correo, contraseña, repetirla y aceptar los términos y la política de privacidad. Tras una autenticación correcta, la aplicación conduce a Inicio y conserva la sesión para posteriores aperturas.

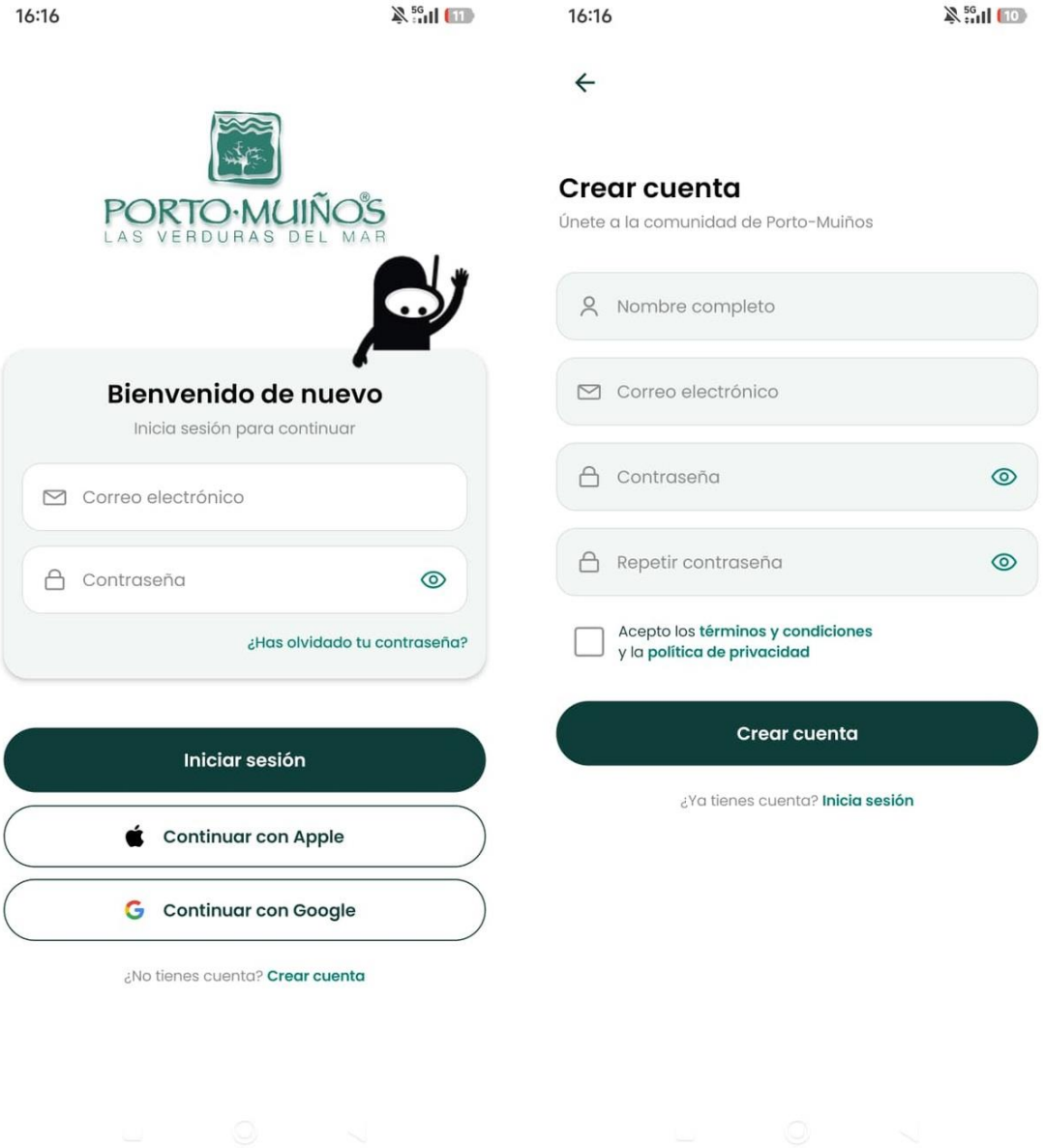


Figura 17. Inicio de sesión y creación de una nueva cuenta en Porto-Muiños App.

Aunque la interfaz conserva el botón «Continuar con Apple» como elemento visual previsto en el diseño, esta modalidad de autenticación no forma parte de las funcionalidades implementadas en la versión final.

B.2. Recuperación de contraseña e Inicio

Si se ha olvidado la contraseña, desde la pantalla de acceso puede seleccionarse «¿Has olvidado tu contraseña?». El usuario introduce el correo asociado a su cuenta y recibe un enlace de recuperación. Al abrirlo, el deep link devuelve el control a la aplicación para establecer una nueva contraseña.

Una vez autenticado, Inicio actúa como punto central de la experiencia. Desde esta pantalla se puede acceder al perfil, favoritos y carrito; realizar búsquedas; abrir AlgaeChef; consultar recetas y productos destacados; y utilizar la barra inferior para navegar entre Inicio, Recetas, AlgaeChef y Tienda. El botón central de la barra abre directamente el escáner QR.

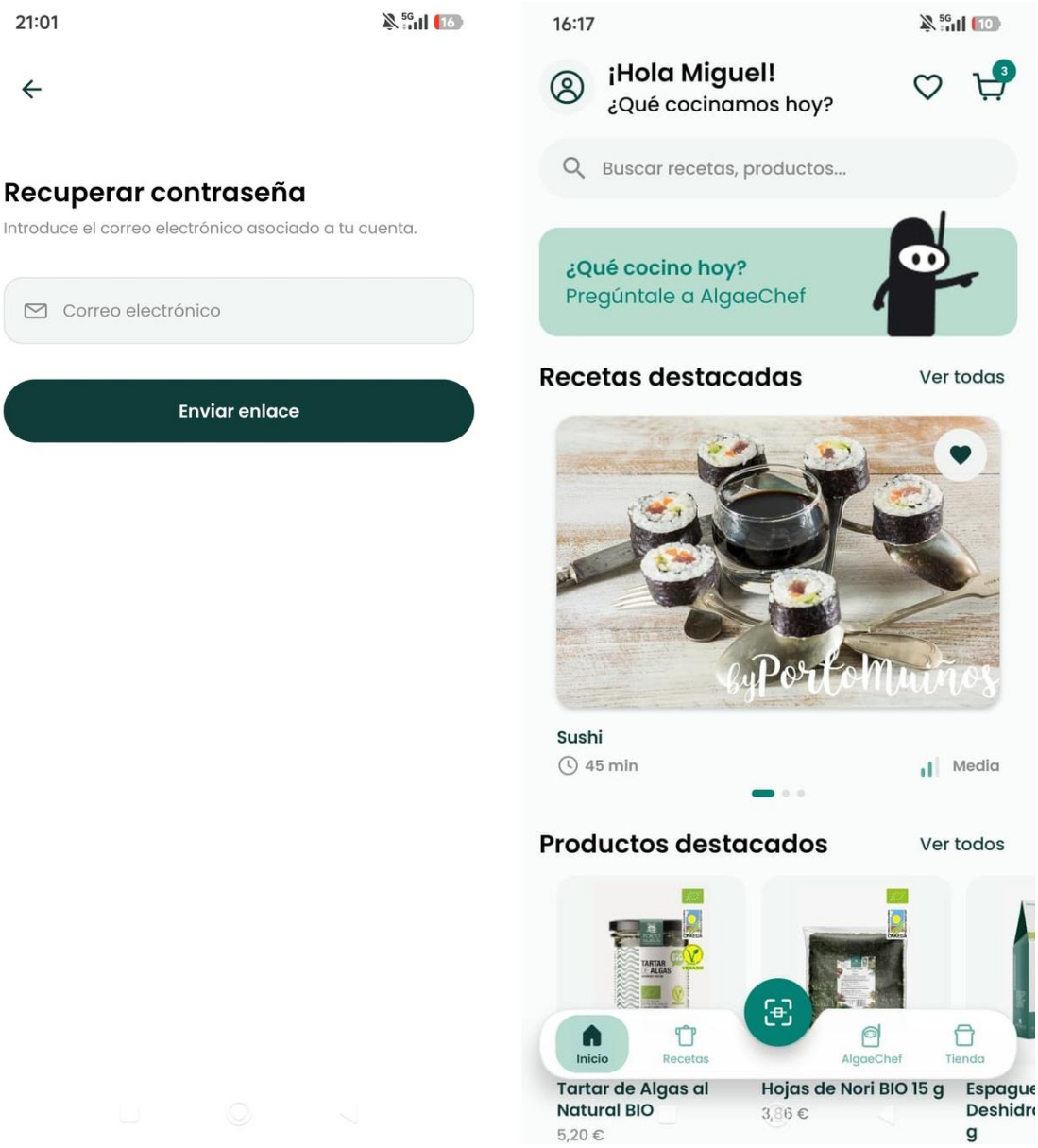


Figura 18. Recuperación de contraseña y pantalla principal de la aplicación.

B.3. Consulta de recetas

La sección Recetas permite explorar el recetario almacenado en Supabase. El buscador filtra por texto y los botones de categoría permiten reducir los resultados.

Cada tarjeta muestra información básica como nombre, duración y dificultad, y puede marcarse como favorita mediante el icono de corazón.

Al abrir una receta se muestra su ficha completa. Las pestañas «Ingredientes» y «Paso a paso» permiten alternar entre la lista de ingredientes y la secuencia de elaboración. En la parte inferior aparecen, cuando existen, los productos Porto-Muiños relacionados, desde los que puede abrirse directamente la ficha comercial correspondiente.

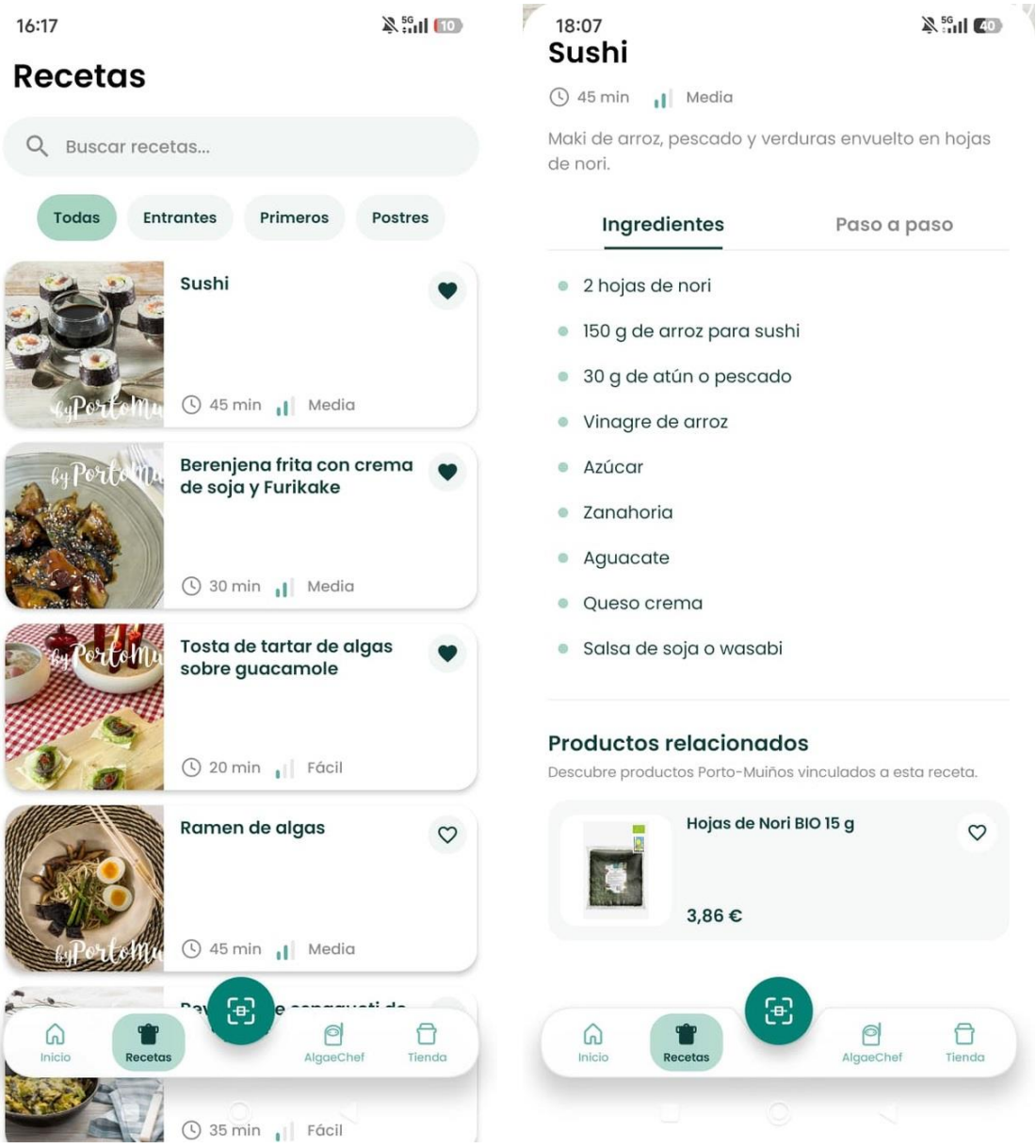


Figura 19. Listado de recetas y detalle con ingredientes y producto relacionado.

B.4. Tienda y detalle de producto

La sección Tienda presenta el catálogo de productos con buscador y filtros por categoría. El botón «+» permite añadir rápidamente una unidad al carrito, mientras que al pulsar sobre un producto se abre su ficha detallada.

En el detalle se muestran nombre, descripción, precio y controles de cantidad. El usuario puede marcar el producto como favorito, añadirlo al carrito, consultar recetas relacionadas y utilizar «Ver en Porto-Muiños» para abrir el contenido oficial correspondiente. El pago y el envío no se realizan dentro de la aplicación, sino en la tienda oficial de Porto-Muiños.

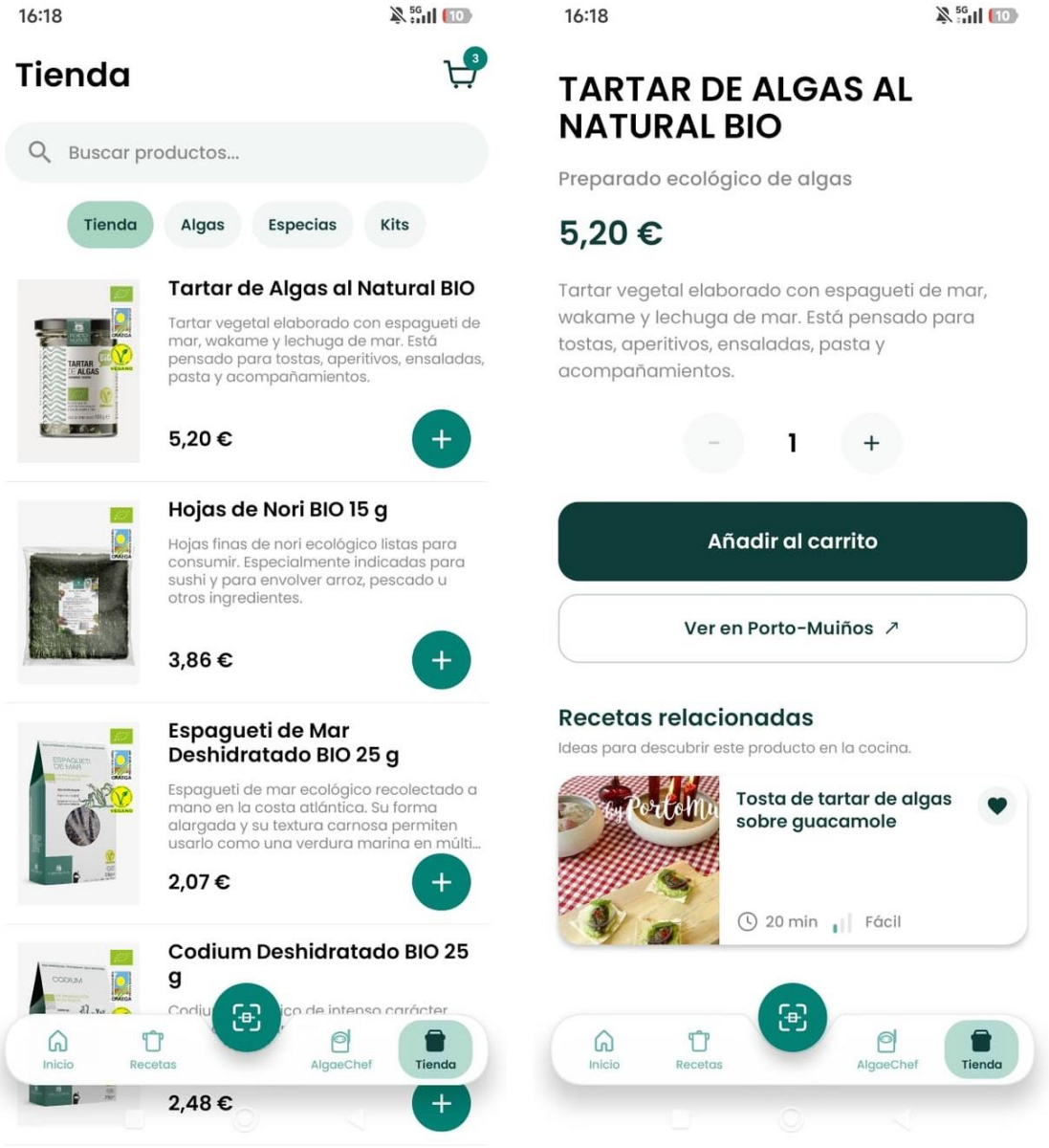


Figura 20. Catálogo de productos y ficha de producto con acciones de compra y recetas relacionadas.

B.5. Escaneo de productos mediante QR

El botón QR central de la navegación inferior abre el escáner. En el primer uso Android solicitará permiso para acceder a la cámara. Una vez concedido, basta con encuadrar un código compatible dentro del área marcada. Cuando el producto es reconocido, la aplicación muestra una confirmación y abre automáticamente su ficha interna. Si el código no puede resolverse, se informa al usuario y puede continuarse mediante búsqueda manual.



Figura 21. Escáner QR detectando un producto real y preparando la apertura de su ficha.

B.6. Uso de AlgaeChef

AlgaeChef puede abrirse desde Inicio, desde su opción en la barra inferior o desde accesos contextuales de la aplicación. El usuario puede escribir una consulta libre o utilizar opciones rápidas como «Tengo pescado», «Algo rápido», «Vegetariano», «Quiero algo ligero» o «Para sorprender».

En esta versión, AlgaeChef funciona como demostrador controlado: interpreta la opción o consulta dentro de los escenarios previstos y devuelve una recomendación explicada

junto con una receta real disponible en la aplicación. La tarjeta de receta puede abrirse directamente para continuar el flujo de navegación.

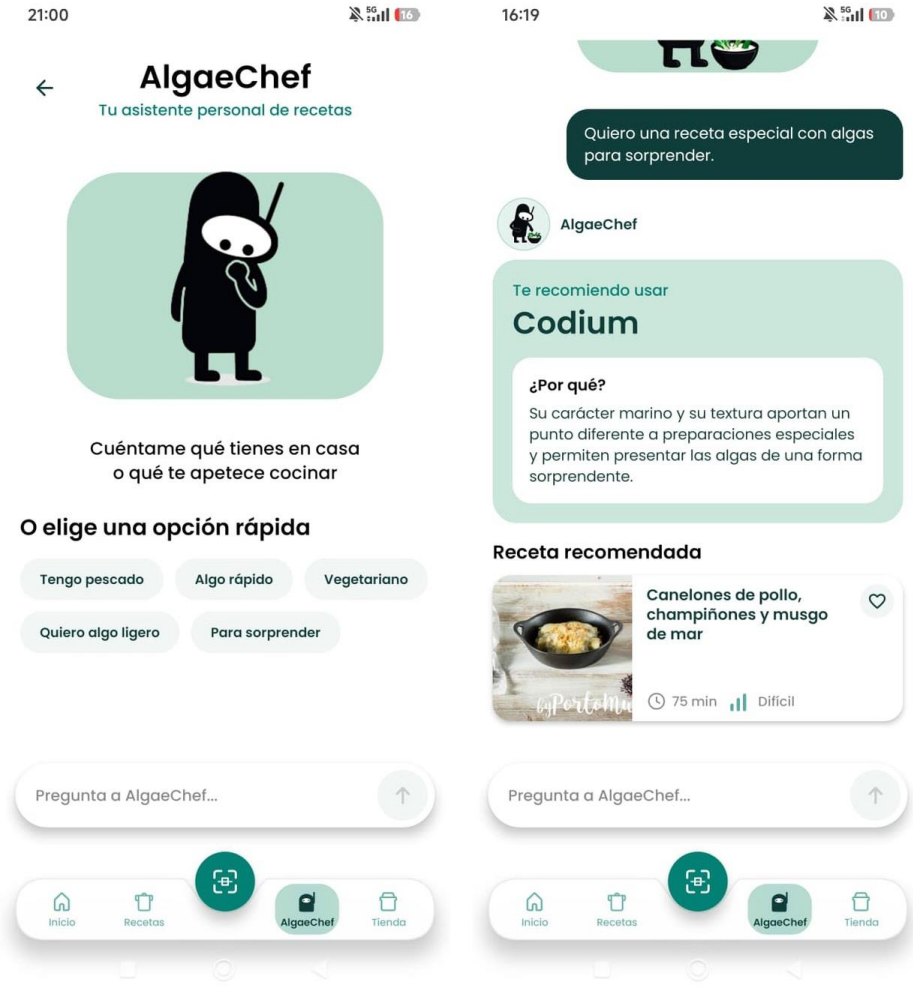


Figura 22. Pantalla inicial de AlgaeChef y ejemplo de recomendación vinculada a una receta real.

B.7. Favoritos

Los iconos de corazón permiten guardar recetas y productos. Los favoritos quedan asociados a la cuenta autenticada y se sincronizan con Supabase. Desde «Mis favoritos» se puede alternar entre las pestañas de Recetas y Productos, abrir cualquiera de los elementos guardados, eliminar favoritos individualmente o utilizar la opción «Vaciar» cuando esté disponible.

B.8. Carrito y continuación de compra

El carrito puede abrirse desde los accesos situados en Inicio, Tienda o el detalle de producto. Cada línea muestra el producto, su precio y la cantidad seleccionada. Los

controles «−» y «+» modifican la cantidad, «Eliminar» retira una línea y «Vaciar» elimina el contenido completo. La parte inferior muestra el total estimado.

El botón «Continuar en Porto-Muiños» abre la tienda oficial para completar el proceso comercial. Por tanto, la aplicación gestiona la selección y persistencia del carrito, mientras que el pago y el envío permanecen en la infraestructura oficial de la empresa.

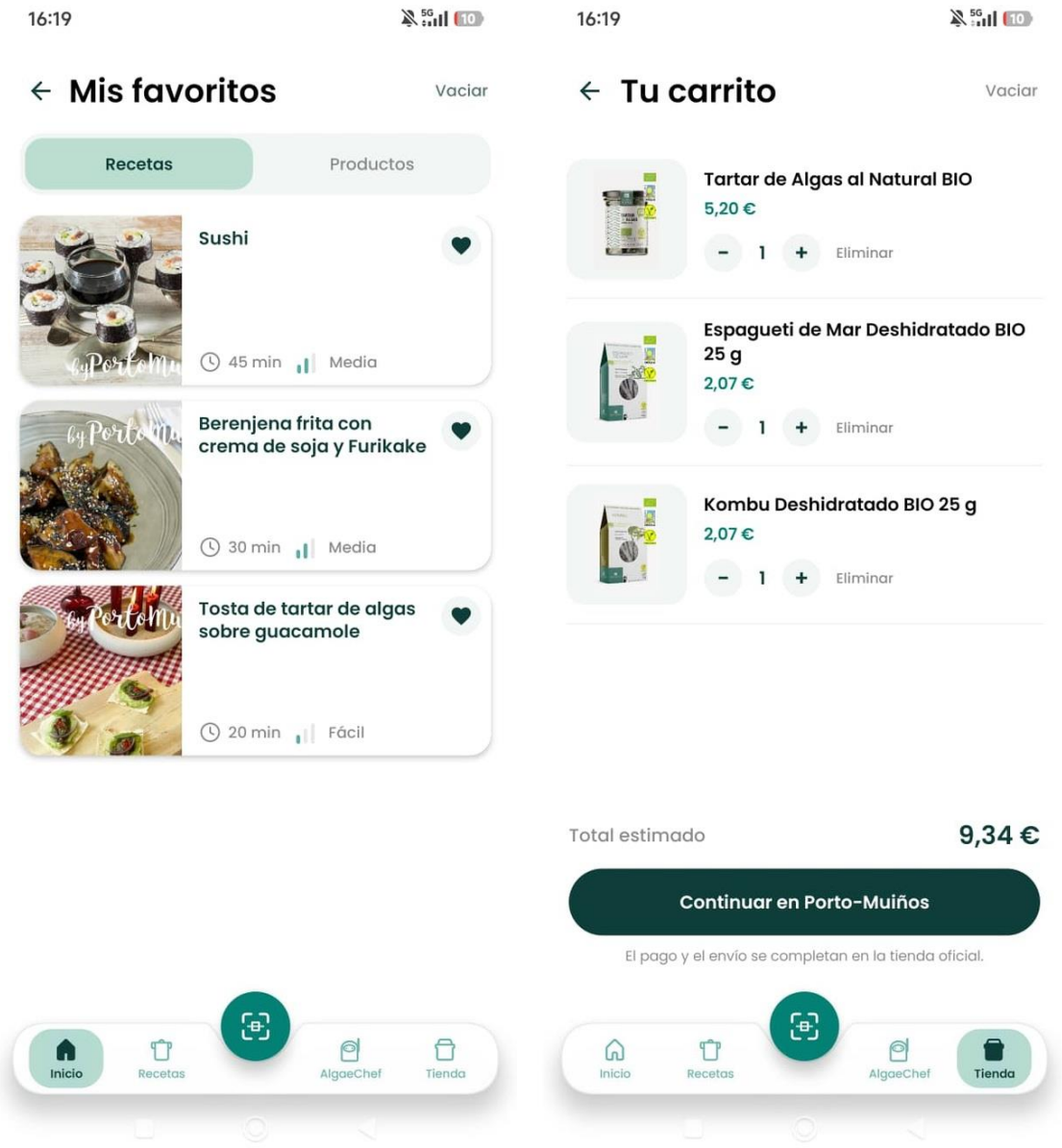


Figura 23. Favoritos persistentes y carrito con cantidades, total estimado y acceso a la tienda oficial.

B.9. Perfil y gestión de la cuenta

El perfil se abre desde el icono situado en la parte superior de Inicio. Muestra el nombre del usuario, su imagen de perfil local y un resumen de actividad. Desde esta pantalla

puede accederse a «Mis favoritos», cambiar la imagen mediante el icono de edición, cerrar sesión y eliminar definitivamente la cuenta.

Las entradas de preferencias alimentarias y alergias/intolerancias forman parte de la evolución prevista del producto y no se consideran funcionalidades operativas dentro del alcance final desarrollado. La eliminación de cuenta, en cambio, sí está implementada y se ejecuta de forma segura mediante una Edge Function autenticada en Supabase.

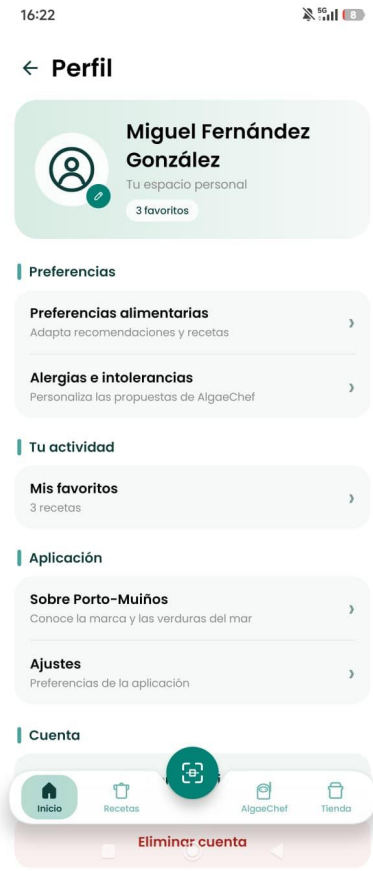


Figura 24. Pantalla de perfil con datos de usuario, favoritos y opciones de cuenta.

B.10. Flujo de uso recomendado

El flujo de uso recomendado consiste en descubrir una receta o un producto, consultar sus relaciones, guardarlo o añadirlo al carrito y continuar hacia la tienda oficial cuando exista intención de compra. El QR permite partir de un envase físico y AlgaeChef ofrece una vía alternativa para llegar a una receta. La navegación inferior mantiene accesibles Inicio, Recetas, AlgaeChef y Tienda durante los recorridos principales, conservando conectado el ciclo producto-receta-recomendación-compra.

Anexo C. Código fuente y aplicación

El código fuente de PortoMuinos App y la versión instalable correspondiente a la entrega final del proyecto se encuentran disponibles en GitHub. La release v1.0 incluye la APK firmada PortoMuinos-v1.0.apk.

Repositorio y APK: https://github.com/miguelfdezglez29/PortoMuinos/releases/tag/v1.0